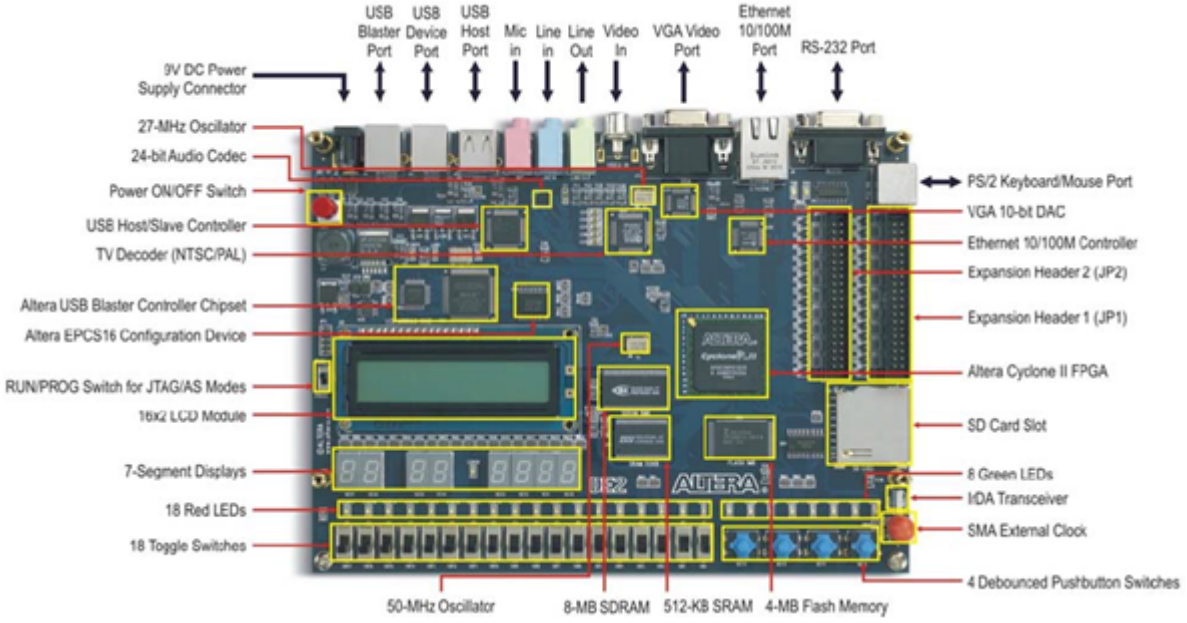
Andreu Mas Viñolas
Sergi Salvà Pibernat

Detailed specifications:

What does the system do?

The system is a two-player hang-man game.

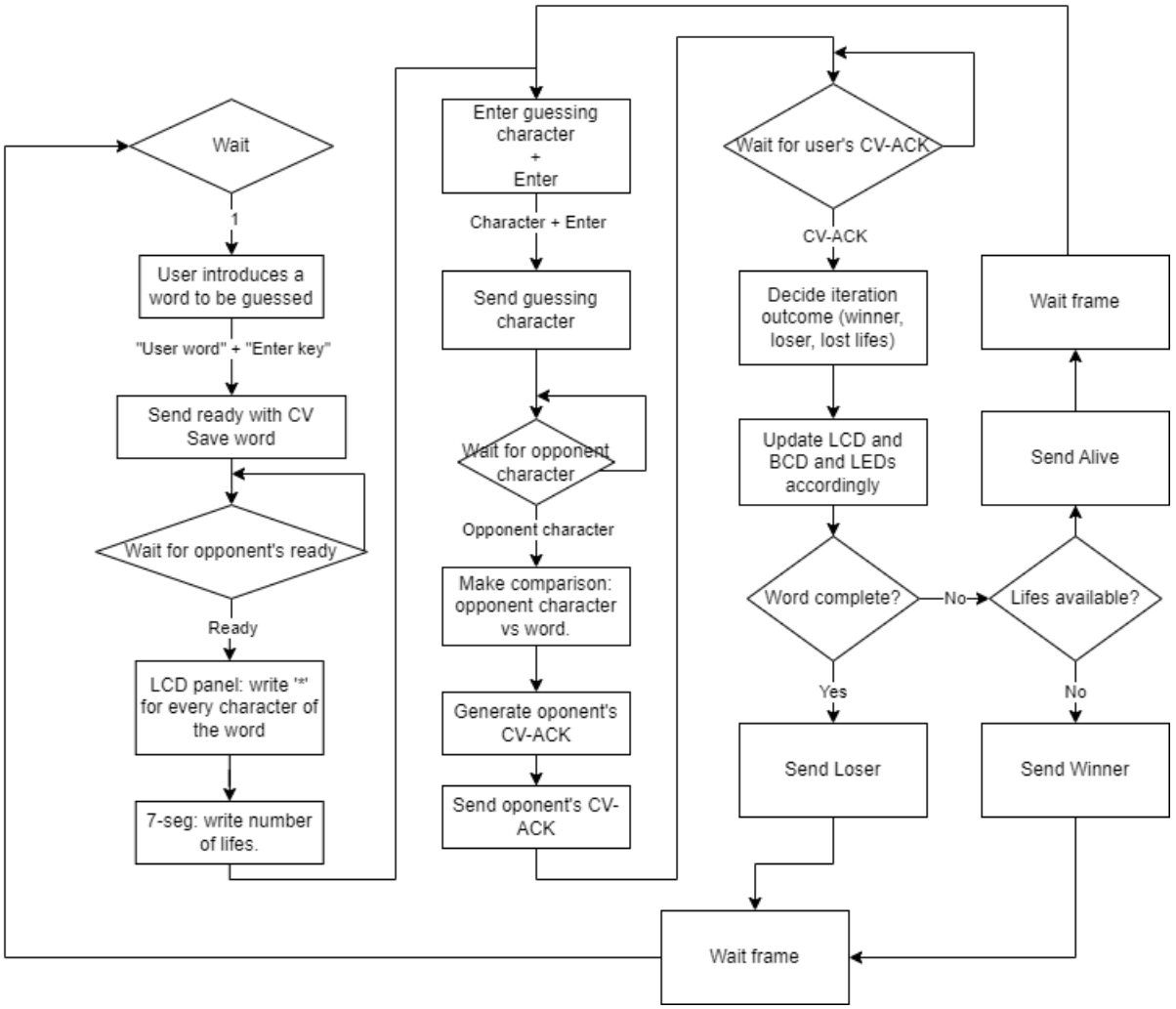
We are going to implement a simultaneous symmetrical double game of hang-man between two players (user and opponent). Each player is in front of a DE2 board. The two boards are connected with a serial cable using the 9 pin serial RS232 port. . The DE2 board is a development board from Altera, which is used to develop designs using the Altera Cyclone ® II 2C35 FPGA device.



High level description of the algorithm

1. User: Presses 1 on the SP2 keyboard to start.
2. User: Introduces the word that needs to be guessed. Maximum 16 characters. *Note: No return (for the moment).*
3. User: Presses *enter* to confirm the word.
4. User: Sends the checking vector (CV) with ones in the used positions to indicate the number of characters the word has.
5. Opponent: send us his CV.
6. User: Introduces a guess letter. Enter to confirm.
7. Opponent: Send us the CV-ACK.
8. User:
    a. If the letter is in the word, the guessed letter is written in the LCD panel in the corresponding positions.
    b. If not, reduce the “lives” in the 7-segments and/or turn off a red LED.
9. Opponent: Same as the user.
10. Once one of the players has guessed all the characters present in the word, this is the winner.
    a. Winner:

   i. All green LEDs on.
   ii. 7-seg shows the word: WIN
b. Loser:
   i. All red LEDs on.
   ii. 7-seg shows the word: LOSE



Communications protocol:

Physically, the boards will communicate through a serial cable, using 3 wires (RX, TX and GND) plugged to the 9 pin RS232 board connector.

The communication protocol is simple. The transmitter must build the frame, which is received and interpreted by the receiver. We will not implement any handshake. We assume that, as we are in a laboratory environment, there will be no communication errors. This means that once the sender has sent the frame, it assumes that it has been correctly received by the receiver. If there is a communication error, the system will not recognize the frame and the game must start over by pressing the RESET.

We design the following frames:

- Ready - Checking Vector (CV)
- Checking Vector ACK (CV-ACK)
- Guessing Character (GC)
- Loser
- Winner

Frame_ID	Frame_Type	CV	CV

Frame_ID	Frame_Type	CV-ACK	CV-ACK

Frame_ID	Frame_Type	GC

Frame_ID	Frame_Type

FRAME_ID: The byte (AA)_hex
FRAME_TYPE:

Ready:	The byte (A1)_hex
CV-ACK:	The byte (B1)_hex
GC:	The byte (C1)_hex
Loser:	The byte (D1)_hex
Winner:	The byte (D2)_hex

DATA:

Ready:	2 bytes
CV-ACK:	2 bytes
GC:	1 byte
Loser:	0 bytes
Winner:	0 bytes

The baud rate must be the same for the user and opponent boards. The input clock frequency is divided by 65536. A bit is going to be sent every 4 clock cycles.

The communication protocol is a ping-pong style. There are three moments while the game is in progress where there is communication between the two boards:

- To send the READY frame when the USER WORD has been introduced.
- To send the USER GUESSING CHARACTER.
- To send the result of the comparison between the USER GUESSING CHARACTER (CV-ACK) and the OPPONENT WORD.
- To inform the opponent that he/she has LOST.
- To inform the opponent that he/she has WON.
- To inform the opponent that he/she is still ALIVE.

When one of these moments arrives, the board sends the adequate frame to the opponent board and then waits for the same frame from the opponent board. Communications are used to synchronize the games in the two boards: the moment when the game starts and the iterations. If one of the boards does not send the frame, there is no time out and the game remains in a stand by mode. The only way to exit from this stand by mode is to RESET the system.

Option1:
At the beginning of the game, each player sends the word. Then when we receive a new character, the opponent knows where it goes in the word.
pros:
The user doesn't need to send the position of the letter → less communications.
cons:
And at the end we need to wait for the other player to finish and compare the lives available for each player.

Option2:
The word is never sent to the opponent. When the opponent sends a guess character, the user answers with a 2-byte vector with 1s in the position where it is. If the character is not present, the two-byte vector is all zero.
pros:
The ACK is the position vector of the guessed character.
Each player manages their own word.
cons:
There are more transmissions.

We finally decided for option two.

User interface:

Inputs:
- QWERTY PS2 keyboard to start the game ("1" key) and to introduce both the user word and the guessing character and their validation ("Enter" key).
- In the board:
    - Push button 0 (P0) to perform a global asynchronous reset.

Outputs:
- In the board:
    - RED LEDS (LR0…17): If all leds are ON, all the lifes are available. When a player runs out of lifes or loses the game all red leds turn OFF. For every life lost one led turns of starting for LR17 all the way to LR0.
    - GREEN LEDS (LG7…0): all leds are ON if the user wins the game.
    - The seven segments will show:
        - H0, H1, H2, H3: either " WIN" or "LOST" to indicate if the user has won or has lost the game.
        - H4, H5: will indicate the number of lifes remaining
    - 16x2 LCD: Will show the word that the user is guessing and the user introduced word to be guessed by the opponent. The characters that the user has guessed will appear at its corresponding position. The characters unguessed will show a hashtag.

Notes:
The LCD Panel permits 16 columns and 2 rows.

The word is guessed character by character. No chance to guess the whole word.
16-bit vector with ones in the positions used for the word: CheckingVector (CV).

Block diagram v1

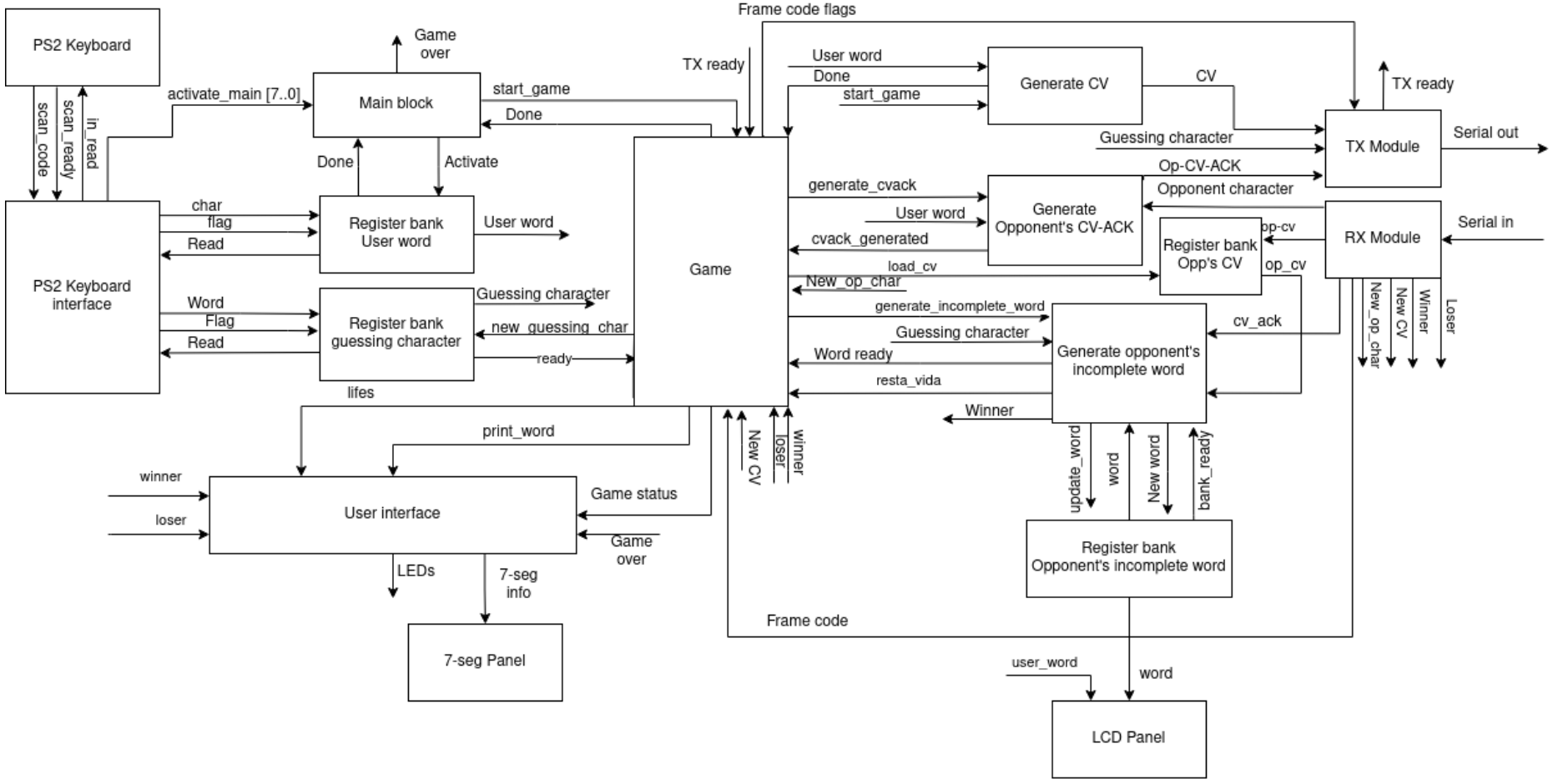
Following there is the list of the different blocks:

- PS2 Keyboard interface:This block will be in charge of driving the external keyboard. The front end with the other blocs of the design will be the word, the character/type of the key pressed and the handshake signals.
- User interface: This block will control the seven segments, leds and LCD screen.
- LCD: This block will control the functioning of the LCD panel showing the different guessed letters of the opponent words.
- hex_disp: this block will control the seven segments that will indicate if we have won or lost and the number of lifes remaining.
- TX Module: this includes the generation of the games depending on the moment of the game and to deal with the physical channel (speed, protocol).
- RX Module: symmetrical as the TX Module.
- Game: once the user code is entered, the game itself has a clear procedure. This
- Generate CV: this block takes in the user word and generates a vector with as many ones as letters has the user word.
- Generate Opponent's CV-ACK: this block takes in the opponent character to guess the user word and generates the CV-ACK vector with ones in the positions where the character is in our letter that is transmitted through the TX Module.
- Generate opponent's incomplete word: this block takes in the user guessing character, the CV, the CV-ACK and the old opponent's incomplete word and adds the guessing character to the positions indicated by the CV-ACK vector. The new opponent's incomplete word is passed on to the Game block and it updates the opponent's incomplete word.
- Main block: like the main function in a software code. Waits for the 1 to be pressed, then activates the storage module for the user code (the software analogy would be to call the function that reads the user code), then activates the game block, which, in turn, activates the the storage module of the guessing word and deals with game itself until the game is over; and in such case, informs the game block.
- Game: Once the user code is entered, the game itself has a clear procedure. This block will interact with the communications block, interact with the two storage blocks (user word and guessing character) and the three generator blocks (Generate CV, Generate Opponent's CV-ACK and Generate opponent's incomplete word).
- Register bank user word: Storage of the user word.
- Register bank guessing character: Storage of the user guessing character.
- Register bank opponent's incomplete word: Storage of the vector of characters of the letters guessed of the opponent's word. If for example, the

opponents word is “hello” and the user has introduced the characters “h” and “l” the opponent’s incomplete word will be: “h*ll*”.

On the following page there is the block diagram for the version 1:

Andreu Mas Viñolas
Sergi Salvà Pibernat

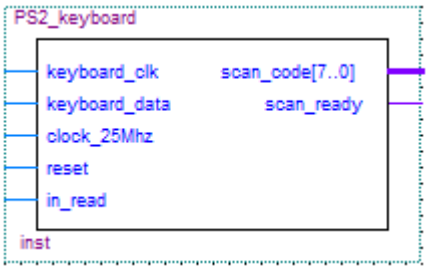


Block diagram v2

All the blocks have in common the inputs CLK (clock, to be connected to the output of the clock divider when tested on board or directly to the clock signal when simulated) and NRST (asynchronous reset, active low, to activate in case the system hangs up).

Note: It has been assumed that all the characters are coded using the ASCII code, and that all the blocks (LCD, Keyboard) also use this codification. There is the possibility that this is not right and that needs a modification.

PS2 Keyboard



This block is an IP. It is used to read the input signals coming from the PS2 keyboard.
It has two clocks, one at the frequency of the game, and another at the frequency at which the keyboard works.
When this IP detects an input from the keyboard, it outputs the scan_code, and scan_ready flag toggles to '1'. Scan_ready will go down when the signal in_read activates.
INPUTS:

- keyboard_clk: clock of the keyboard.
- keyboard_data: awakening signal coming from the PS2 keyboard interface. Active, high, synchronous. This signal must be active for one clock period.
- clock_25Mhz: clock at 25MHz
- reset: synchronous reset, active high.
- in_read: awakening signal coming from the PS2 keyboard interface block. Active, high, synchronous. This signal must be active for one clock period. Resets the scan ready handshake signal.

OUTPUTS:

- scan_code[7..0]: character scanned by the keyboard.
- scan_ready: flag for a handshake signal needed to ensure that a new scan code is read in and processed only once. it is set whenever a new scan code is received.

PS2 Keyboard interface

This block serves as interference between the PS2 keyboard IP and the game itself. When the user presses a key in the keyboard, the IP block will output a scan code and the signal scan_ready will be activated. This block detects it, reads the code of the pressed key (key-down and key-up), converts it to ASCII, and toggles the flag. When this flag is toggled to '1', it indicates to other blocks that there is a new char to read coming from the keyboard.
INPUTS:

- scan_ready: Awakening that is on if the keyboard is ready.
- ack_flag: to put down the flag indicating that the character has been read.
- new_code: activator for the block
- scan_code[7..0]: Character read by the keyboard IP.

OUTPUTS:

- flag: flag that indicates when a character is ready to be read.
- in_read: signal used to give feedback to the PS2 keyboard IP.
- keycode[7..0]: character entered by the user.

User interface

This block is the main controller for all the user interface elements such as the leds, the lcd panel and the seven segment display. In this block the conversion from the four bit lifes number to the conversion to be understood by the hex_disp module. It also changes the output of the different elements depending on the value of game_status.
All individual input and output signals are active high.
INPUTS:

- word_ready_in: awakening signal that indicates to the LCD module that it needs to update the content of the LCD screen.
- lifes[3..0]: The number of lifes to be displayed in the 7-segment display.
- game_status[1..0]: 00 if "win", 01 if "lose", 10 if the game continues.
- game_over: from the main block. Active when the game is over.
- lost: flag to indicate that the user has lost.
- winner: flag to indicate that the user has won.

OUTPUTS:

- Red LEDs (LR16..0): the signal will activate the red leds indicating the number of lifes available or they will be all on if the user has lost.
- Green LEDs (LG7..0): the signal will activate the green leds. This signal will be activated when the player wins.
- 7seg[29..0]: The information to be displayed in the 7-segment display (win, lose, lifes)

lcd

This module is the control for the LCD display used in the altera DE2 board. As an input it has the two words that we show at the LCD display such as the user word on the first line and the opponent's incomplete word is at the second line.
INPUTS:

- user_word[127..0]: word introduced by the user to be guessed by the opponent player.
- word[127..0]: this word is the opponent's incomplete word that we have guessed so far, to be displayed at the first line of the LCD.

OUTPUTS:

- on: flag, active high that indicates if the LCD is ON or OFF.
- rw: flag, active high that indicates if we read or write.
- rs: flag, active high that indicates if we are in the setup process or showing data of the LCD.
- e: flag, active high for enabling the LCD.
- lcd_data[7..0]: data signals for lcd.

hex_disp

This block is just a mapping for the input signals to be translated to the different segment segment displays

The signals connected to the seven segments are active low.

INPUTS:

- sev_seg[29..0]: The information to be displayed in the 7-segment display (win, lose, lifes)

OUTPUTS:

- HEX0, HEX1, HEX2, HEX3: These outputs should be routed to the FPGA pins connected to the seven segment displays, named in the DE2 board as HEX0, 1, 2, 3. They will show the text “ WIN” or “ LOST”.
- HEX4, HEX5: These outputs should be routed to the FPGA pins connected to the seven segment displays, named in the DE2 board as HEX4, 5. They will show the number of lifes available to the user.

BCD7seg

This block just converts a 5 bit input to a 7 bit output to be understood by the seven segment lights.

TX Protocol

This block serves as an interface between the game and the transmission IP block. It also holds the communication protocol and transmits when desired.

The outputs are connected to the transmission IP block. This block sends the corresponding signals to the IP to properly send the data.

INPUTS:

- CV[15..0]: The opponent’s generated CV to be transmitted.
- op_cvack[15..0]: The opponent’s generated CV-ACK to be transmitted.
- send_gc: signal to send the guessing character.
- send_loser: signal to inform the opponent that he/she has lost.
- send_winner: signal to inform the opponent that he/she has won.
- tx_empty: flag that when is ‘0’ indicates that the comms channel is free and the TX module can transmit safely.

- send_ready: signal to send the 'ready' message with the CV.
- send_op_cv_ack: signal to send the opponent's calculated cv-ack
- guessing_char[7..0]: The character guessed by the user that will be transmitted to the opponent.

OUTPUTS:

- tx_data[7..0]: Byte to be transmitted
- ready_to_tx: This flag indicates that the TX module is ready to transmit. '1' if ready to transmit, '0' otherwise.
- send: signal that activates the TX module IP.

RX Protocol

This block acts as the receiver protocol. Once initiated it waits for the frame id message to arrive, once it has arrived it waits for the frame type and once the frame type is received it checks the frame type. If the type contains data, the system reads the extra bytes. Depending on the frame received it activates the corresponding flag and it updates the data valued outputs. Once all this process is done it waits for another frame id message to arrive.

INPUTS:

- ByteRX[7..0]: Serial input. Connected to the TX output of the opponent board.
- frame_read: Flag that indicates that the game module has read the previous frame.
- rx_new: flag that indicates that the rx_module has received a neu frame.

OUTPUTS:

- op_cv[15..0]: The received CV from the opponent.
- cv_ack[15..0]: The received CV-ACK from the opponent.
- op_char[7..0]: The received character from the opponent.
- new_op_char: Flag that indicates that there is a new received message that is a character. '1' if a new character has been received, '0' otherwise.
- new_cv: Flag that indicates that there is a new received message that is a CV. '1' if a new CV has been received, '0' otherwise.
- new_op_cv: Flag that indicates that an opponent cv has been received.
- loser: flag that indicates that a loser frame has been received.
- winner: flag that indicates that a winner frame has been received.

Game

This block is the core of the game. It is awakened when the user has entered his/her word to be guessed.

The block is in charge of coordinating all the blocks by sending and receiving signals to them. This block doesn't do any processing nor computations, it only deals with the coordination of the game and asks other blocks to do their tasks.

INPUTS:

- start_game: Awakening signal from the main block.
- cv_done: Signal that indicates that the CV has been generated.
- tx_ready: indicates that the TX module is ready to transmit.

- opponent_ready: Flag that indicates that there is a new received message that is a CV. ‘1’ if a new CV has been received, ‘0’ otherwise.
- done_reg_gc: Signal activated by the “register bank guessing character” when he has sent the guessing character.
- cvack_generated: Signal that indicates that the CV-ACK has been generated.
- word_ready: Awakening signal that informs “game” that the new incomplete word is already in the “Register bank opponent’s incomplete word”.
- winner: Flag either received or local that indicates that the user has won.
- loser: Flag received that indicates that the user has lost.
- resta_vida: signal that indicates that the game value of lifes has to decrease by one unit.
- new_op_char: Flag that indicates that there is a new received message that is a character. ‘1’ if a new character has been received, ‘0’ otherwise.
- new_op_cv: Flag that indicates that there is a new op_cv saved at the register.

OUTPUTS:

- activate_reg_gc: Awakening signal to update the “Register bank guessing character” block with the new character.
- load_cv: flag that indicates to the register_bank_opponent_cv to load the opponent cv to the register.
- send_gc: flag that indicates to the TX module to send a guessing character frame.
- generate_cvack: Signal that activates the CV-ACK generator.
- send_op_cvack: signal that activates the cv_ack_generator module.
- activate_gen_inc_word: signal that activates the gen_op_inc_word module.
- print_word: flag that activates the user_interface module.
- led_green: this signal activates a green led that indicates when the user can introduce a guessing character.
- led_red: this signal activates a red led that indicates when you have to wait for the opponent and you cannot introduce a new character.
- game_done: flag that indicates to the main module that the game has finished.
- send_loser: flag that indicates to the TX module to send a loser frame.
- send_winner: flag that indicates to the TX module to send a winner frame.
- generate_cv: Signal that activates the CV generator.
- send_ready:: flag that indicates to the TX module to send a ready frame.
- game_veredict[1..0]: 00 if “win”, 01 if “lose”, 10 if the game continues.
- lifes[3..0]: number of remaining lifes.
- frame_received: flag that indicates to the RX module that a received frame has been read.

Generate CV

This block generates the CV of the user. Recall that the CV (Checking Vector) is a 16-bit vector that has as many ‘1’s as letters the word has. For instance, for the word “hello”, the calculated CV would be: ‘0000000000011111’.
INPUTS:

- word[127..0]: The user word needed to compute the opponent’s CV.
- activate: Awakening signal sent by the “game” block to activate the “generate CV” block.

OUTPUTS:

- CV[15..0]: The generated CV to be transmitted to the opponent.
- done: Flag that indicates the “game” module that the CV has been generated and transmitted to the “TX module”.

CV-ACK-generator

This block generates the cv-ack that is sent to the opponent. It waits for the generate_cvack flag to be active and once it is active it computes the cv-ack and then it outputs it. To compute the cv-ack it compares the character that the opponent has guessed and it compares it to the user word character by character. If the character it coincides with a letter of the user word it introduces a 1 to the op_cvack if it doesn’t coincide it introduces a 0.
INPUTS

- opponent_char[7..0]: opponent’s received character.
- user_word[127..0]: The user word needed to compute the opponent’s CV-ACK
- generate_cvack: Awakening signal sent by the “game” block to activate the “generate opponent’s CV-ACK” block.

OUTPUTS:

- op_cvack[15..0]: The opponent’s generated CV-ACK.
- cvack_generated: Awakening signal that indicates the “game” module that the CV-ACK has been generated and the TX module has it.

Generate opponent’s incomplete word

This block generates the word that the user is guessing from the opponent and that is displayed in the second line of the LCD display. It waits for the awakening signal gen_inc_word.
Once awakened it checks if the cvack received from the opponent is equal to 0, then the user has missed the guess and it activates the signal resta_vida and returns to the initial waiting state.
If the cvack received is different from 0, then the user has guessed a character from the opponent word. From there it computes the incomplete word and then it saves it to the register bank.
To compute the incomplete word it goes through the rx_cvack and if in that position it is 1 then in that position of the new_word it introduces the guess_char saved on its register.

Then it computes an internal cv and then compares it to the initial opponent cv and if it is the same, then the flag winner activates.
To compute the internal cv, it goes through the new_word and compares every character of the word with x"00", so to check if that position is occupied as it is done at the generate_cv module.

INPUTS:

- gen_inc_word: Awakening signal sent by the "game" block to activate the "generate opponent's incomplete word" block.
- rb_inc_word_rdy:
- old_word[127..0]: The incomplete word saved from the previous iteration (not considering the new guessing character!)
- guess_char[7..0]: The guessing character written by the user.
- rx_cvack[15..0]: The received cv ack from the opponent user.
- op_cv[15..0]: The opponent cv that indicates the length of the opponent guessing word.

OUTPUTS:

- word_ready: Awakening signal that informs the "game" block that the new word is already in the "register bank opponent's incomplete word".
- new_word[127..0]: The incomplete word updated with the new guessing character.
- resta_vida: Signal that indicates that the user has not guessed the character and it is needed to decrease one life of the user.
- update_word: Flag that indicates to the register to load new_word.
- winner: Signal that indicates that the user has won.

Main block

This block is the one that coordinates the beginning of the game, the end of the game, and the beginning of the next game.

INPUTS:

- char: Character from the keyboard interface. If 1, the game starts.
- user_word_ready: Awakening signal that confirms that the user's word has been saved.
- game_done: Signal that informs the block "main block" that the game is over. (this is the signal coming from the game module)

OUTPUTS:

- get_word: Awakening signal. Synchronous signal, active high. Duration: 1 clock period. Indicates to the register bank user word block that can read a new word.
- game_over: Flag that is '1' if the game is over and '0' if a game is going on.
- start_game: Awakening signal that starts the "game" module.
- ack_flag: signal used to inform the word register that the user's word has been read by the main block.

Register bank user word

This block saves the user's word. This register is loaded at the very beginning of the game, and it is kept unchanged for the whole game until the user wins or loses.

INPUTS:

- new_word: Awakening signal activated by the MAIN block.
- char_flag: Flag from the keyboard block that indicates that a character is ready to be read.
- char[7..0]: Character written by the user. It is connected to the output of the keyboard block.

OUTPUTS:

- bank_ready: Active high. The signal is active when the register bank is ready to be awakened and to store a new word. When the input new_word is activated, this signal deactivates, and it is activated again when the user presses the key "enter".
- key_read: ACK signal for the keyboard when a key has been read by this block.
- user_word[127..0]: word written by the user.

Register bank guessing character

This block saves the guessing characters that the user enters throughout the game.

INPUTS:

- new_char: Awakening signal indicating that a new char is ready to be read in the keyboard block.
- char_flag: Flag signal from the keyboard block.
- char[7..0]: character written by the user. Connected to the PS2 keyboard interface block's output.

OUTPUTS:

- bank_ready: Active high. The signal is active when the register bank is ready to be awakened and to store a new character. When the input new_character is activated, this signal deactivates, and it is activated again when the user presses the key "enter".
- key_read: ACK signal for the keyboard when a key has been read by this block.
- out_char[7..0]: character written by the user and stored in the register. Signal connected to the Game block.

Register bank opponent's incomplete word

This module consists of a register bank for the opponent's incomplete word. It waits for the awakening signal f_new to be activated to load the new_word input to the register and output its value. When that is complete it activates the flag bank_ready and it goes back to wait for f_new to be activated.

INPUTS:

- new_word[127..0]: The part of the word that has been guessed by the user.
- f_new: Flag that indicates to the register to load new_word.

OUTPUTS:

- word[127..0]: The incomplete word that the user guessed. This signal is connected directly to the LCD panel.
- bank_ready: Flag that indicates that the register has loaded correctly the incomplete word.

Register bank opponent's CV

This module consists of a register bank for the opponent's cv. It waits for the awakening signal load_cv to be activated to load the op_cv input to the register and output its value. Then it goes back to wait for load_cv to be activated.

INPUTS:

- op_cv[15..0]: The opponent's CV received (by the opponent at the beginning of the game).
- load_cv: Awakening signal that, when activated, loads whatever it is in the op_cv[15..0] input into the register.

OUTPUTS:

- cv_out[15..0]: The opponent's CV saved in the register.
- bank_ready: Flag that indicates that the register has loaded correctly the incomplete word.

Work plan

Delivery 1: Minimum 35% of the final work (17/11/2022)

- State diagram, VHDL for the PS2 Keyboard, PS2 Keyboard interface, User interface, LCD Panel and 7-seg Panel blocks.
- Simulation and test for the PS2 Keyboard and LCD screen.
- Algorithm, state diagram, VHDL and simulation for the register banks of the user word, guessing character, and opponent's incomplete word.

Delivery 2: Minimum 60% of the final work (01/12/2022)

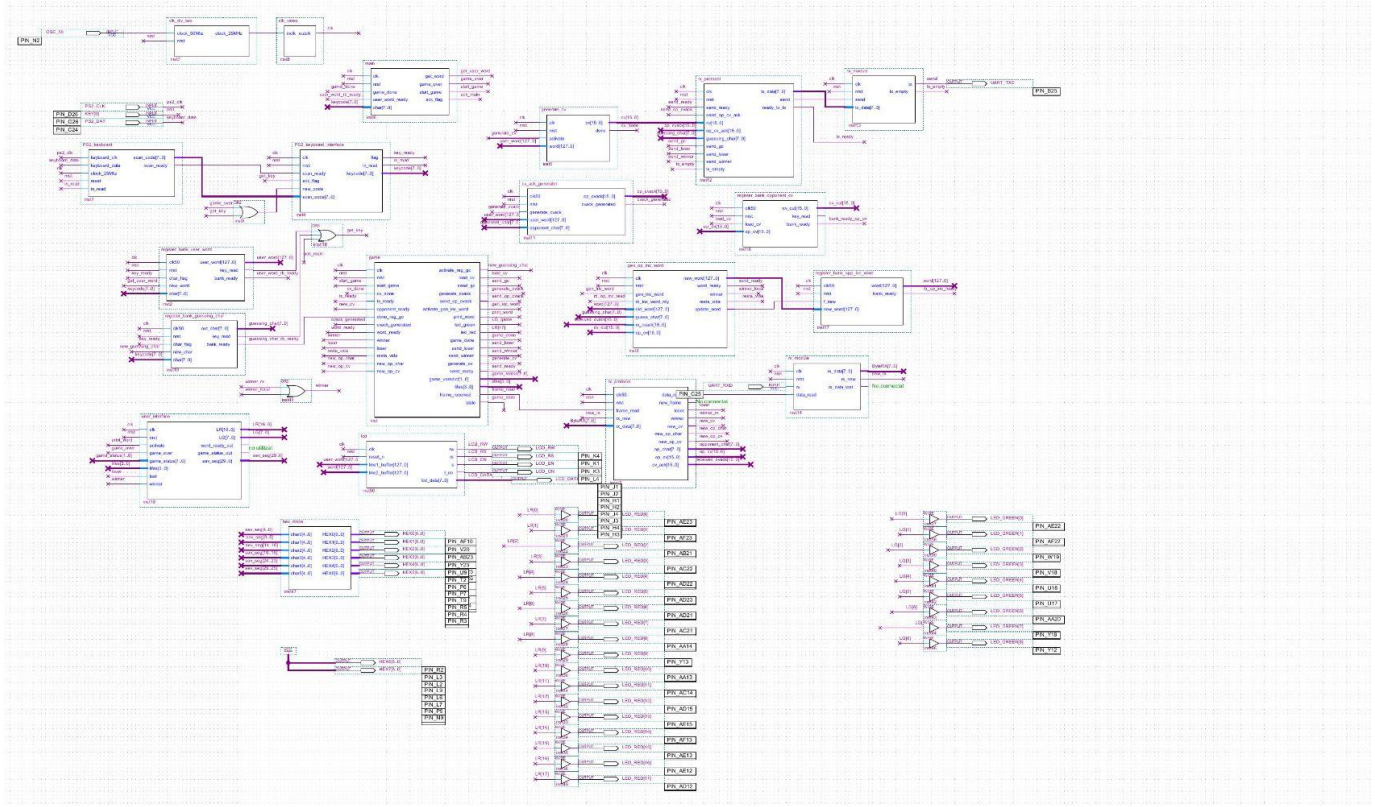
- Algorithm, state diagram, VHDL and simulation for the:
    - Main block (Andreu)
    - Game block (Andreu i Sergi)
    - Generate CV (Andreu)
    - Generate CV-ACK (Sergi)
    - Generate opponent's incomplete word (Sergi)

Board test: (22/12/2022)

- Algorithm, state diagram, VHDL and simulation for the:
    - TX Module
    - RX Module
- Game simulation
- Board game

Final schematic:

Date: December 31, 2022 hangman.bdf* Project: hangman



Page 1 of 1 Revision: hangman

Project Folder

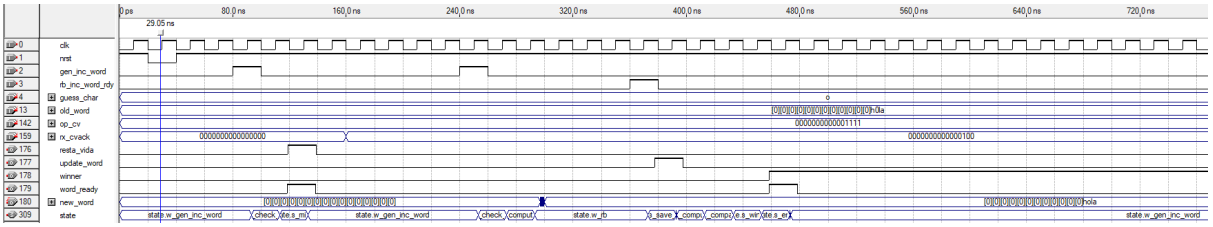
In the following link you can find the project folder containing all the documentation and the runnable vhd and bdf files of the project.
Hangman Project Andreu Mas Viñolas and Sergi Salvà Pibernat

Andreu Mas Viñolas
Sergi Salvà Pibernat

Annex 1: Simulations descriptions

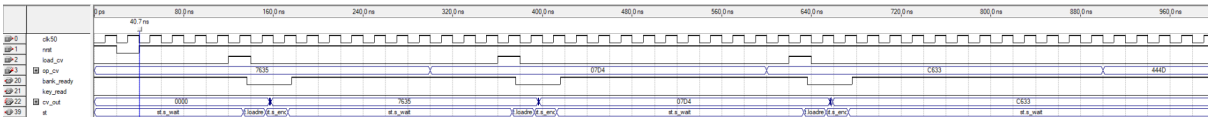
File name: gen_op_inc_word.vwf

In this file we test the correct functionality of the gen_op_inc_word module, introducing manually all the inputs.



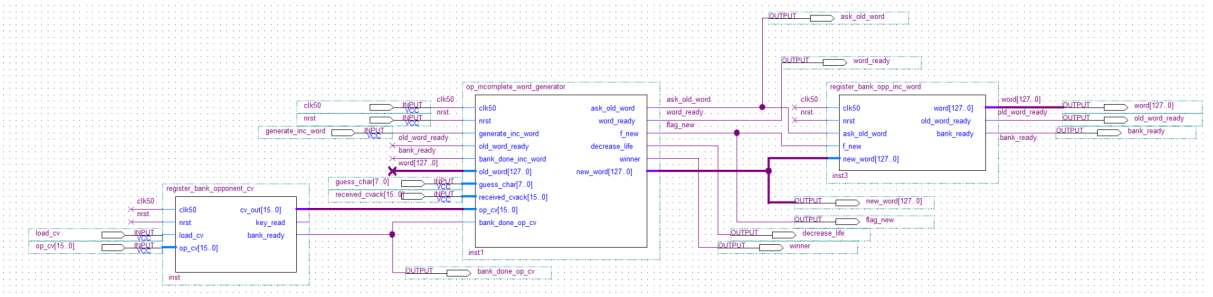
File name: register_bank_opponent_cv.vwf

In this file we test the correct functionality of the register_bank_opponent_cv module, introducing manually the inputs and observing the correct loading of the cv at the register.

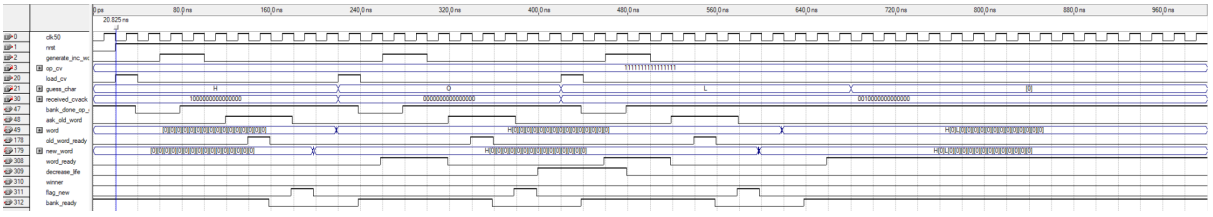


File name: opponent_incomplete_word_test.vwf

To test the correctness of the generation of the incomplete word we designed the following block scheme where we test the correctness of the relation between the opponent cv register bank, the op_incomple_word_generator module and the opponent incomplete word register bank as it can be seen in the following image:

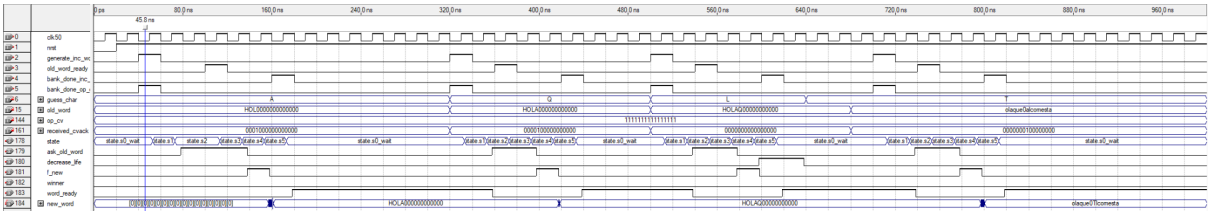


In the opponent_incomplete_word_test.vwf file we check that each module works properly together.



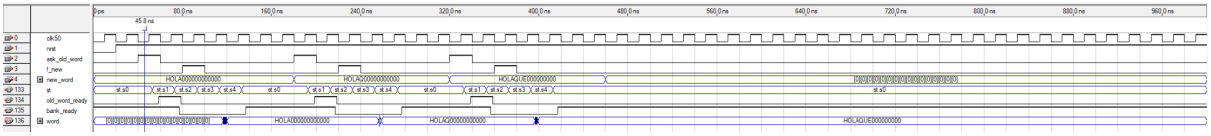
File name: op_incomplete_word_generator_sim.vwf

In this file we test the correct functionality of the op_incomplete_word_generator module, introducing manually all the inputs and verifying that we obtain the desired result.



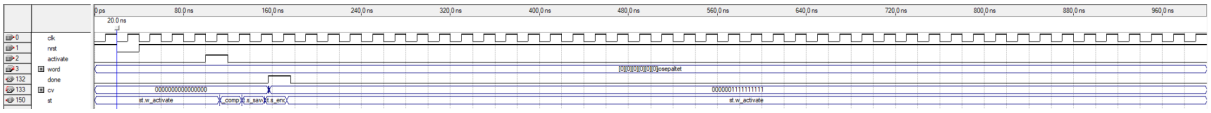
File name: register_bank_opp_inc_word_sim.vwf

In this file we test the correct functionality of the register_bank_opp_inc_word module, introducing manually the inputs and observing the correct loading of the guessed opponent incomplete word at the register.



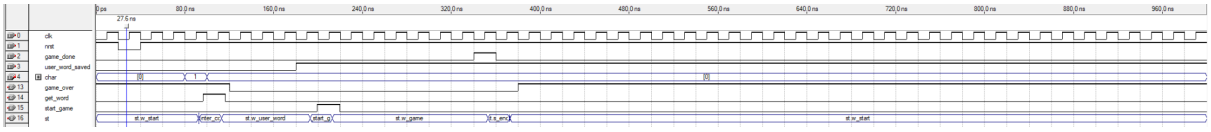
File name: generate_cv.vwf

In this file we test the correct functionality of the cv generator module, introducing manually all the inputs and verifying that we obtain the desired result.



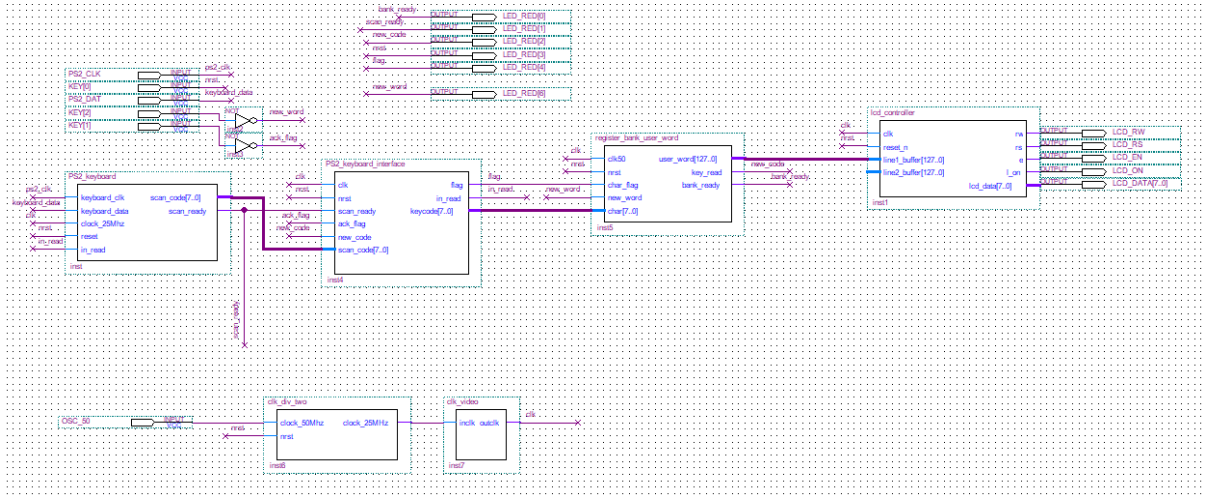
File name: main.vwf

In this file we test the correct functionality of the main module, introducing manually all the inputs and verifying that we obtain the desired result.



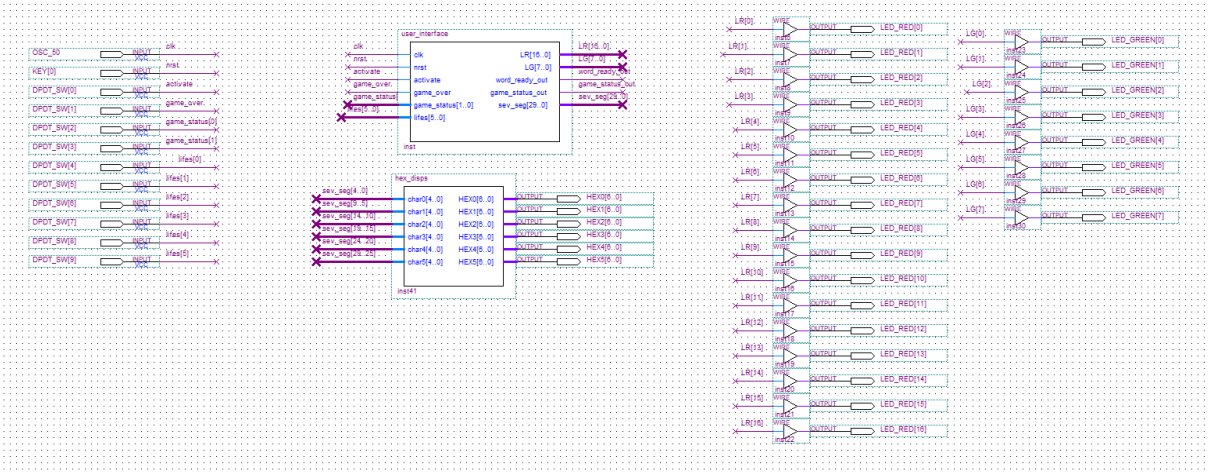
File name: keyboard_test.bdf

In this block file we created the testing file for the keyboard.



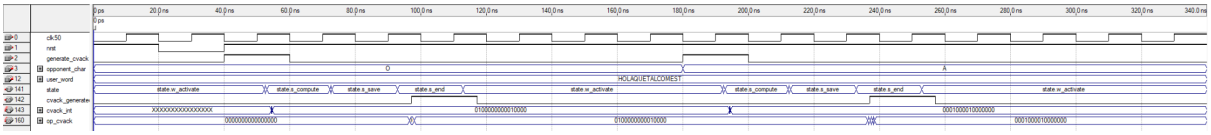
File name: user_interface_tes.bdf

In this block file we created the testing infrastructure to test the correctness of the user interface and hex displays functionality as well as de LCD and leds.



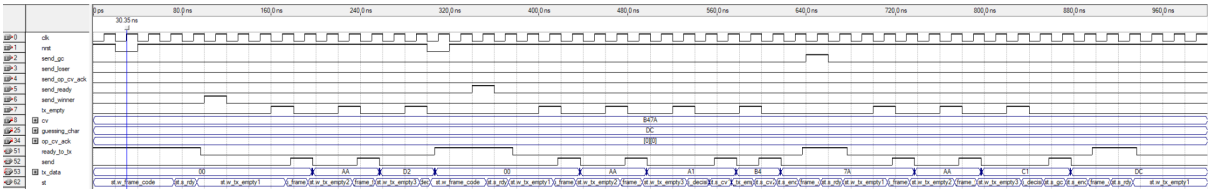
File name: cv_ack_generator_sim.vwf:

In this file we test the correct functionality of the cv_ack_generator module by introducing manually the inputs and observing the correctly generated cv_ack.



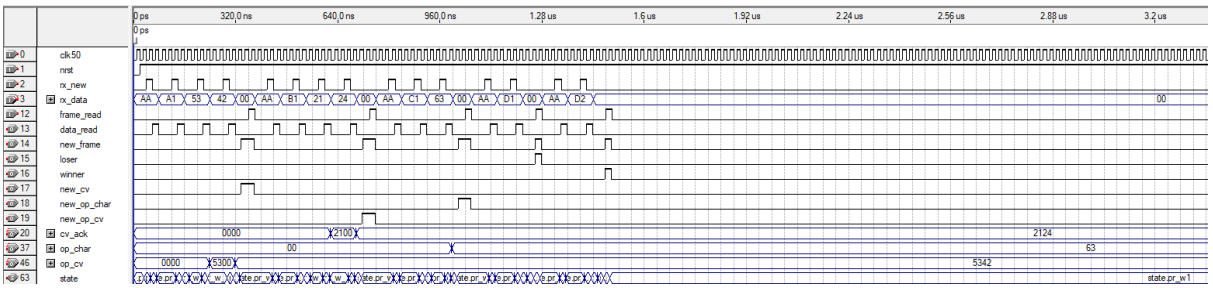
File name: tx_protocol.vwf:

In this file we test the correct functionality of the tx_protocol that we had designed.



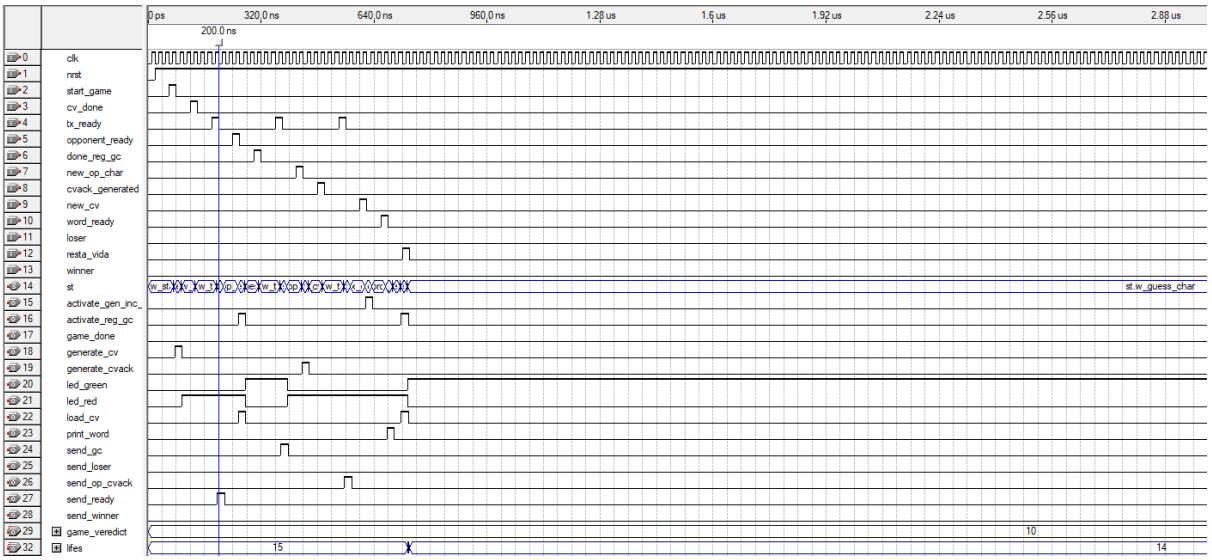
File name: rx_protocol_sim.vwf:

In this file we thet the correct functionality of the rx_protocol that we had designed by introducing the inputs manually and visualizing the activation of the different flags.



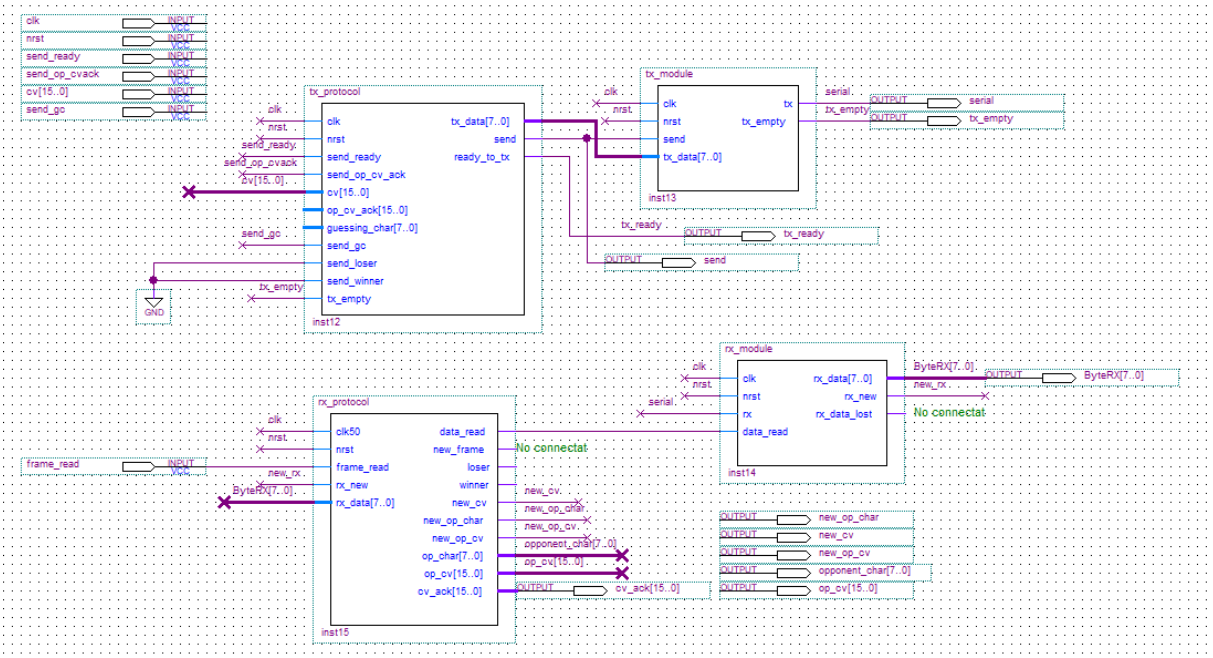
File name: game_sim.vwf:

In this file we tested the game module and its correct generation of the corresponding output signals.



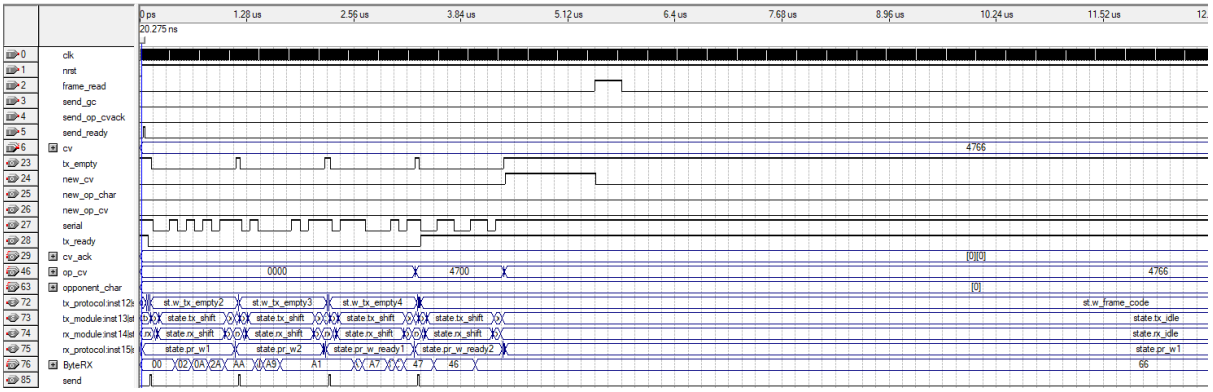
File name: tx_rx_test.bdf:

In this file we create a block structure to test the correctness of transmission and reception functionality.



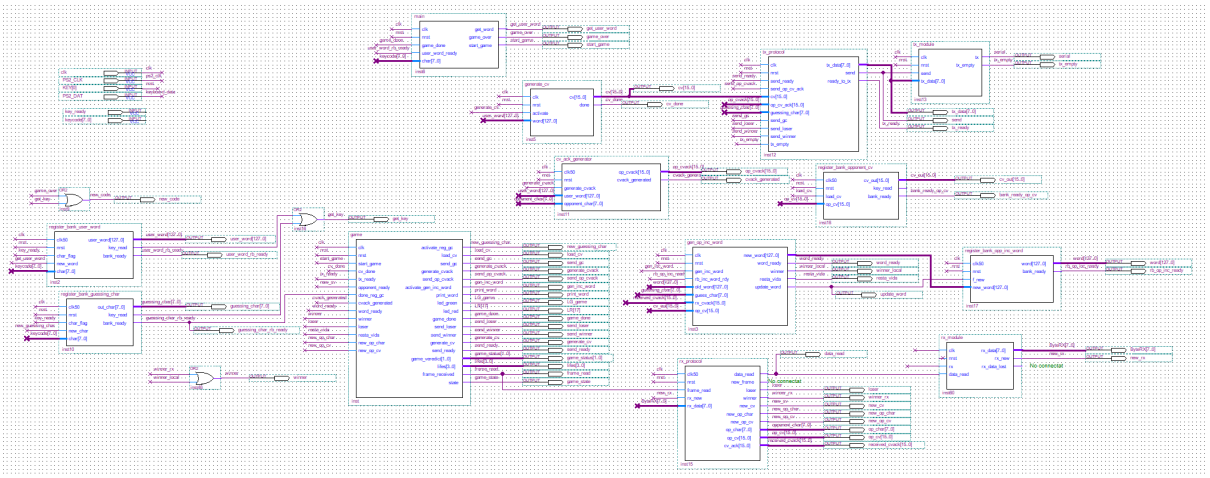
File name: tx_rx_test_sim.vwf:

In this file we test the correct sending and receiving of the different generated frames and the activation of the corresponding outputs and flags.



File name: hangman.vwf:

To test the correctness of the running game we designed the following block scheme where we connect the output of the TX at the input of the RX as it can be seen in the following image:



In the hangman_sim.vwf file we check that each module works properly together.

Annex2:

Date: December 31, 2022 cv_ack_generator.vhd Project: hangman

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;
4
5   entity cv_ack_generator is
6       port(   clk50, nrst, generate_cvack: in std_logic;
7               user_word: in std_logic_vector (127 downto 0);
8               opponent_char: in std_logic_vector(7 downto 0);
9               op_cvack: out std_logic_vector (15 downto 0);
10              cvack_generated: out std_logic);
11  end cv_ack_generator;
12
13  architecture arc of cv_ack_generator is
14      type state_type is (w_activate, s_compute, s_save, s_end);
15      signal state    : state_type;
16
17      signal ld       : std_logic;
18      signal cvack_int    : std_logic_vector (15 downto 0);
19
20  begin
21  process(clk50, nrst) begin
22      if nrst = '0' then
23          state <= w_activate;
24      elsif clk50'event and clk50 = '1' then
25          case state is
26              when w_activate =>
27                  if (generate_cvack = '1') then
28                      state <= s_compute;
29                  end if;
30              when s_compute =>
31                  state <= s_save;
32              when s_save =>
33                  state <= s_end;
34              when s_end =>
35                  state <= w_activate;
36          end case;
37      end if;
38  end process;
39
40
41  process(user_word, opponent_char, cvack_int)
42  begin
43      if state = s_compute then
44          for count in 0 to 15 loop
45              if(user_word(((count*8)+7) downto (count*8)) =
    opponent_char) then
46                  cvack_int(count) <= '1';
47              else
48                  cvack_int(count) <= '0';
49              end if;
50          end loop;
51      end if;
52  end process;
53
54
55  process(clk50,ld,nrst)
56  begin
57      if nrst = '0' then
58          op_cvack <= x"0000";
```

```
59        elsif (clk50'event and clk50 = '1') then
60            if ld = '1' then
61                op_cvack <= cvack_int;
62            end if;
63        end if;
64    end process;
65
66    ld <= '1' when state = s_save else '0';
67    cvack_generated <= '1' when state = s_end else '0';
68
69    end arc;
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;
4   use ieee.numeric_std.all;
5   use ieee.std_logic_arith.all;
6
7   entity game is
8   port(   clk, nrst, start_game, cv_done, tx_ready, opponent_ready :
     in std_logic;
9           done_reg_gc, cvack_generated, word_ready, winner, loser,
    resta_vida, new_op_char, new_op_cv: in std_logic;
10          activate_reg_gc, load_cv, send_gc, generate_cvack,
    send_op_cvack: out std_logic;
11          activate_gen_inc_word, print_word, led_green, led_red,
    game_done: out std_logic;
12          send_loser, send_winner, generate_cv, send_ready : out
    std_logic;
13          game_veredict: out std_logic_vector(1 downto 0);
14          lifes : out std_logic_vector(3 downto 0);
15          frame_received, state: out std_logic);
16
17  end game;
18
19  architecture arc of game is
20      type state_type is (w_start, s_gen_cv, w_cv, w_tx1,
    s_send_ready, w_op_ready, s_guess_char, w_guess_char,
21                          w_tx2, s_send_gc, w_op_char, s_gen_cvack,
    w_cvack, w_tx3, s_send_cvack, w_rx_cvack, s_gen_iw,
22                          w_word_ready, s_lcd, s_game_veredict,
    w_tx4, w_tx5, s_send_winner, s_send_loser);
23      signal st : state_type;
24      signal J_G, K_G, J_R, K_R : std_logic;
25      signal int_lifes : integer;
26      signal buf_lifes : std_logic_vector(3 downto 0);
27      signal int_game_veredict: std_logic_vector(1 downto 0);
28
29  begin
30
31      -- States machine
32      process(clk, nrst) begin
33          if nrst = '0' then
34              st <= w_start;
35          elsif clk'event and clk = '1' then
36              case st is
37                  when w_start =>
38                      if(start_game = '1') then
39                          st <= s_gen_cv;
40                      end if;
41                  when s_gen_cv =>
42                      st <= w_cv;
43                  when w_cv =>
44                      if(cv_done = '1') then
45                          st <= w_tx1;
46                      end if;
47                  when w_tx1 =>
48                      if(tx_ready = '1') then
49                          st <= s_send_ready;
50                      end if;
51                  when s_send_ready =>
```

```
52                      st <= w_op_ready;
53                  when w_op_ready =>
54                      if(opponent_ready = '1') then
55                          st <= s_guess_char;
56                      end if;
57                  when s_guess_char =>
58                      st <= w_guess_char;
59                  when w_guess_char =>
60                      if(done_reg_gc = '1') then
61                          st <= w_tx2;
62                      end if;
63                  when w_tx2 =>
64                      if(tx_ready = '1') then
65                          st <= s_send_gc;
66                      end if;
67                  when s_send_gc =>
68                      st <= w_op_char;
69                  when w_op_char =>
70                      if(new_op_char = '1') then
71                          st <= s_gen_cvack;
72                      end if;
73                  when s_gen_cvack =>
74                      st <= w_cvack;
75                  when w_cvack =>
76                      if(cvack_generated = '1') then
77                          st <= w_tx3;
78                      end if;
79                  when w_tx3 =>
80                      if (tx_ready ='1') then
81                          st <= s_send_cvack;
82                      end if;
83                  when s_send_cvack =>
84                      st <= w_rx_cvack;
85                  when w_rx_cvack =>
86                      if(new_op_cv ='1') then
87                          st <= s_gen_iw;
88                      end if;
89                  when s_gen_iw =>
90                      st <= w_word_ready;
91                  when w_word_ready =>
92                      if(word_ready = '1') then
93                          st <= s_lcd;
94                      end if;
95                  when s_lcd =>
96                      st <= s_game_veredict;
97                  when s_game_veredict =>
98                      if(int_game_veredict = "10") then
99                          st<= s_guess_char;
100                     elsif(int_game_veredict = "00") then
101                         st<= w_tx4;
102                     else
103                         st<= w_tx5;
104                     end if;
105                 when w_tx4 =>
106                     if (tx_ready ='1') then
107                         st <= s_send_loser;
108                     end if;
109                 when w_tx5 =>
110                     if (tx_ready ='1') then
```

```
111                         st <= s_send_winner;
112                     end if;
113                 when s_send_loser =>
114                     st <= w_start;
115                 when s_send_winner =>
116                     st <=w_start;
117             end case;
118         end if;
119     end process;
120
121
122     -- Combinacional circuit lifes
123     process(clk, nrst, resta_vida)
124     begin
125         if nrst = '0' then
126             int_lifes <= 16;
127         elsif (clk'event and clk = '1' and resta_vida = '1') then
128             int_lifes <= int_lifes - 1;
129         end if;
130     end process;
131
132     -- Combinacional circuit game_veredict
133     process(clk, nrst, winner, loser)
134     begin
135         if nrst = '0' then
136             int_game_veredict <= "10";
137         elsif (clk'event and clk = '1') then
138             if winner = '1' then
139                 int_game_veredict <= "00";
140             elsif loser = '1' or int_lifes = 1 then
141                 int_game_veredict <= "01";
142             else
143                 int_game_veredict <= "10";
144             end if;
145         end if;
146     end process;
147
148     -- JK flip-flops
149     process(clk,nrst,J_G,K_G)
150     begin
151         if nrst = '0' then
152             led_green <= '1';
153         elsif (clk'event and clk = '1') then
154             if J_G = '1' then
155                 led_green <= '1';
156             elsif K_G = '1' then
157                 led_green <= '0';
158             end if;
159         end if;
160     end process;
161
162     process(clk,nrst,J_R,K_R)
163     begin
164         if nrst = '0' then
165             led_red <= '0';
166         elsif (clk'event and clk = '1') then
167             if J_R = '1' then
168                 led_red <= '1';
169             elsif K_R = '1' then
```

```
170                 led_red <= '0';
171             end if;
172         end if;
173     end process;
174
175 -- Output activations
176 game_done <= '1' when st = s_send_loser or st = s_send_winner else
     '0';
177 generate_cv <= '1' when st = s_gen_cv else '0';
178 J_R <= '1' when st = s_gen_cv or st = s_send_gc else '0';
179 K_G <= '1' when st = s_gen_cv or st = s_send_gc else '0';
180 send_ready <= '1' when st = s_send_ready  else '0';
181 K_R <= '1' when st = s_guess_char else '0';
182 J_G <= '1' when st = s_guess_char else '0';
183 load_cv <= '1' when st = s_guess_char else '0';
184 activate_reg_gc <= '1' when st = s_guess_char else '0';
185 send_gc <= '1' when st = s_send_gc else '0';
186 generate_cvack <= '1' when st = s_gen_cvack else '0';
187 send_op_cvack <= '1' when st = s_send_cvack else '0';
188 activate_gen_inc_word <= '1' when st = s_gen_iw else '0';
189 print_word <= '1' when st = s_lcd else '0';
190 send_winner <= '1' when st = s_send_winner else '0';
191 send_loser <= '1' when st = s_send_loser else '0';
192 frame_received <= '1' when st = s_guess_char or st = s_gen_cvack
    or st = s_gen_iw else '0';
193 state <= '1' when st = w_op_ready else '0';
194
195 --direct connection
196 buf_lifes <=  std_logic_vector(to_unsigned(int_lifes, buf_lifes'
    length)-1);
197 --buf_lifes <= conv_std_logic_vector(int_lifes, buf_lifes'length);
198 lifes <= buf_lifes;
199 game_veredict <= int_game_veredict;
200 end arc;
201
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;
4
5   entity gen_op_inc_word is
6       port(   clk, nrst, gen_inc_word, rb_inc_word_rdy: in std_logic;
7               old_word: in std_logic_vector (127 downto 0);
8               guess_char: in std_logic_vector(7 downto 0);
9               rx_cvack, op_cv: in std_logic_vector (15 downto 0);
10              new_word: out std_logic_vector (127 downto 0);
11              word_ready, winner, resta_vida, update_word: out
    std_logic);
12  end gen_op_inc_word;
13
14  architecture arc of gen_op_inc_word is
15      type state_type is (w_gen_inc_word, s_check_cvack, s_miss,
    s_compute_word, w_rb, s_save_word, s_compute_cv, s_compare_cv,
    s_winner, s_end);
16      signal state    : state_type;
17      signal J_WIN, K_WIN : std_logic;
18      signal compute_word : std_logic;
19      signal cv_int : std_logic_vector (15 downto 0);
20      signal word_buff : std_logic_vector (127 downto 0);
21
22  begin
23  -- states machine
24  process(clk, nrst) begin
25      if nrst = '0' then
26          state <= w_gen_inc_word;
27      elsif clk'event and clk = '1' then
28          case state is
29              when w_gen_inc_word =>
30                  if (gen_inc_word = '1') then
31                      state <= s_check_cvack;
32                  end if;
33              when s_check_cvack =>
34                  if (rx_cvack = x"0000") then
35                      state <= s_miss;
36                  else
37                      state <= s_compute_word;
38                  end if;
39              when s_miss =>
40                  state <= w_gen_inc_word;
41              when s_compute_word =>
42                  state <= w_rb;
43              when w_rb =>
44                  if (rb_inc_word_rdy = '1') then
45                      state <= s_save_word;
46                  end if;
47              when s_save_word =>
48                  state <= s_compute_cv;
49              when s_compute_cv =>
50                  state <= s_compare_cv;
51              when s_compare_cv =>
52                  if (cv_int = op_cv) then
53                      state <= s_winner;
54                  else
55                      state <= s_end;
56                  end if;
```

```
57              when s_winner =>
58                  state <= s_end;
59              when s_end =>
60                  state <= w_gen_inc_word;
61          end case;
62      end if;
63  end process;
64
65
66  -- Put the new character into the incomplete word
67  process(clk,nrst,compute_word)
68  begin
69      if nrst = '0' then
70          word_buff <= x"00000000000000000000000000000000";
71      elsif (clk'event and clk = '1') then
72          if compute_word = '1' then
73              word_buff <= old_word;
74              for count in 0 to 15 loop
75                  if(rx_cvack(count)= '1') then
76                      word_buff(((count*8)+7) downto (count*8)) <=
    guess_char;
77                  end if;
78              end loop;
79          end if;
80      end if;
81  end process;
82
83
84  -- JK FF to control the winner flag
85  process(clk,nrst,J_WIN,K_WIN)
86  begin
87      if nrst = '0' then
88          winner <= '0';
89      elsif (clk'event and clk = '1') then
90          if J_WIN = '1' then
91              winner <= '1';
92          elsif K_WIN = '1' then
93              winner <= '0';
94          end if;
95      end if;
96  end process;
97
98  -- Combinacional circuit to compute
99  -- the CV from the user word
100 process(clk,nrst,word_buff,cv_int)
101 begin
102     if nrst = '0' then
103         cv_int <= x"0000";
104     elsif (clk'event and clk = '1') then
105         if state = s_compute_cv then
106             if word_buff(7 downto 0) = x"00" then
107                 cv_int(0) <= '0';
108             else
109                 cv_int(0) <= '1';
110             end if;
111             if word_buff(15 downto 8) = x"00" then
112                 cv_int(1) <= '0';
113             else
114                 cv_int(1) <= '1';
```

```
115             end if;
116             if word_buff(23 downto 16) = x"00" then
117                 cv_int(2) <= '0';
118             else
119                 cv_int(2) <= '1';
120             end if;
121             if word_buff(31 downto 24) = x"00" then
122                 cv_int(3) <= '0';
123             else
124                 cv_int(3) <= '1';
125             end if;
126             if word_buff(39 downto 32) = x"00" then
127                 cv_int(4) <= '0';
128             else
129                 cv_int(4) <= '1';
130             end if;
131             if word_buff(47 downto 40) = x"00" then
132                 cv_int(5) <= '0';
133             else
134                 cv_int(5) <= '1';
135             end if;
136             if word_buff(55 downto 48) = x"00" then
137                 cv_int(6) <= '0';
138             else
139                 cv_int(6) <= '1';
140             end if;
141             if word_buff(63 downto 56) = x"00" then
142                 cv_int(7) <= '0';
143             else
144                 cv_int(7) <= '1';
145             end if;
146             if word_buff(71 downto 64) = x"00" then
147                 cv_int(8) <= '0';
148             else
149                 cv_int(8) <= '1';
150             end if;
151             if word_buff(79 downto 72) = x"00" then
152                 cv_int(9) <= '0';
153             else
154                 cv_int(9) <= '1';
155             end if;
156             if word_buff(87 downto 80) = x"00" then
157                 cv_int(10) <= '0';
158             else
159                 cv_int(10) <= '1';
160             end if;
161             if word_buff(95 downto 88) = x"00" then
162                 cv_int(11) <= '0';
163             else
164                 cv_int(11) <= '1';
165             end if;
166             if word_buff(103 downto 96) = x"00" then
167                 cv_int(12) <= '0';
168             else
169                 cv_int(12) <= '1';
170             end if;
171             if word_buff(111 downto 104) = x"00" then
172                 cv_int(13) <= '0';
173             else
```

```
174                 cv_int(13) <= '1';
175             end if;
176             if word_buff(119 downto 112) = x"00" then
177                 cv_int(14) <= '0';
178             else
179                 cv_int(14) <= '1';
180             end if;
181             if word_buff(127 downto 120) = x"00" then
182                 cv_int(15) <= '0';
183             else
184                 cv_int(15) <= '1';
185             end if;
186         end if;
187     end if;
188 end process;
189 
190 resta_vida <= '1' when state = s_miss else '0';
191 word_ready <= '1' when state = s_miss or state = s_end else '0';
192 compute_word <= '1' when state = s_compute_word else '0';
193 update_word <= '1' when state = s_save_word else '0';
194 J_WIN <= '1' when state = s_winner else '0';
195 K_WIN <= '1' when state = s_check_cvack else '0';
196 new_word <= word_buff;
197 
198 end arc;
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;
4
5   entity generate_cv is
6       port(   clk, nrst, activate: in std_logic;
7               word: in std_logic_vector (127 downto 0);
8               cv: out std_logic_vector (15 downto 0);
9               done : out std_logic);
10  end generate_cv;
11
12  architecture arc of generate_cv is
13      type state_type is (w_activate, s_compute, s_save, s_end);
14      signal st : state_type;
15      signal ld       : std_logic;
16      signal compute  : std_logic;
17      signal cv_int   : std_logic_vector (15 downto 0);
18      signal found    : std_logic;
19  begin
20
21      -- States machine
22      process(clk, nrst) begin
23          if nrst = '0' then
24              st <= w_activate;
25          elsif clk'event and clk = '1' then
26              case st is
27                  when w_activate =>
28                      if (activate = '1') then
29                          st <= s_compute;
30                      end if;
31                  when s_compute =>
32                      st <= s_save;
33                  when s_save =>
34                      st <= s_end;
35                  when s_end =>
36                      st <= w_activate;
37              end case;
38          end if;
39      end process;
40
41
42      -- Combinacional circuit to compute
43      -- the CV from the user word
44      process(clk, nrst, word,cv_int)
45      begin
46          if nrst = '0' then
47              cv_int <= x"0000";
48          elsif clk'event and clk = '1' then
49              if st = s_compute then
50                  if word(7 downto 0) = x"00" then
51                      cv_int(0) <= '0';
52                  else
53                      cv_int(0) <= '1';
54                  end if;
55                  if word(15 downto 8) = x"00" then
56                      cv_int(1) <= '0';
57                  else
58                      cv_int(1) <= '1';
59                  end if;
```

```
60                  if word(23 downto 16) = x"00" then
61                      cv_int(2) <= '0';
62                  else
63                      cv_int(2) <= '1';
64                  end if;
65                  if word(31 downto 24) = x"00" then
66                      cv_int(3) <= '0';
67                  else
68                      cv_int(3) <= '1';
69                  end if;
70                  if word(39 downto 32) = x"00" then
71                      cv_int(4) <= '0';
72                  else
73                      cv_int(4) <= '1';
74                  end if;
75                  if word(47 downto 40) = x"00" then
76                      cv_int(5) <= '0';
77                  else
78                      cv_int(5) <= '1';
79                  end if;
80                  if word(55 downto 48) = x"00" then
81                      cv_int(6) <= '0';
82                  else
83                      cv_int(6) <= '1';
84                  end if;
85                  if word(63 downto 56) = x"00" then
86                      cv_int(7) <= '0';
87                  else
88                      cv_int(7) <= '1';
89                  end if;
90                  if word(71 downto 64) = x"00" then
91                      cv_int(8) <= '0';
92                  else
93                      cv_int(8) <= '1';
94                  end if;
95                  if word(79 downto 72) = x"00" then
96                      cv_int(9) <= '0';
97                  else
98                      cv_int(9) <= '1';
99                  end if;
100                 if word(87 downto 80) = x"00" then
101                     cv_int(10) <= '0';
102                 else
103                     cv_int(10) <= '1';
104                 end if;
105                 if word(95 downto 88) = x"00" then
106                     cv_int(11) <= '0';
107                 else
108                     cv_int(11) <= '1';
109                 end if;
110                 if word(103 downto 96) = x"00" then
111                     cv_int(12) <= '0';
112                 else
113                     cv_int(12) <= '1';
114                 end if;
115                 if word(111 downto 104) = x"00" then
116                     cv_int(13) <= '0';
117                 else
118                     cv_int(13) <= '1';
```

```
119                     end if;
120                     if word(119 downto 112) = x"00" then
121                         cv_int(14) <= '0';
122                     else
123                         cv_int(14) <= '1';
124                     end if;
125                     if word(127 downto 120) = x"00" then
126                         cv_int(15) <= '0';
127                     else
128                         cv_int(15) <= '1';
129                     end if;
130                 end if;
131             end if;
132         end process;
133 
134 
135         -- Register to save the CV
136         process(clk,ld,nrst)
137         begin
138             if nrst = '0' then
139                 cv <= x"0000";
140             elsif (clk'event and clk = '1') then
141                 if ld = '1' then
142                     cv <= cv_int;
143                 end if;
144             end if;
145         end process;
146 
147 
148     compute <= '1' when st = s_compute else '0';
149     ld <= '1' when st = s_save else '0';
150     done <= '1' when st = s_end else '0';
151 
152     end arc;
```

```
1  ------------------------------------------------------------------
   -------------
2  --
3  --   FileName:         lcd_controller.vhd
4  --   Dependencies:     none
5  --   Design Software:  Quartus II 32-bit Version 11.1 Build 173
   SJ Full Version
6  --
7  --   This file is a derivation of the implementation found at
8  --
   https://eewiki.net/download/attachments/4096079/lcd_controller.vhd?
   version=3&modificationDate=1339620193283&api=v2
9  --
10 --   Version History
11 --   Version 1.0 6/2/2006 Scott Larson
12 --     Initial Public Release
13 --    Version 2.0 6/13/2012 Scott Larson
14 --      Version 3.0 10/01/2017 Mayur Panchal
15 --
16 --   CLOCK FREQUENCY: to change system clock frequency, change
   Line 65
17 --
18 --   LCD INITIALIZATION SETTINGS: to change, comment/uncomment
   lines:
19 --
20 --   Function Set
21 --      2-line mode, display on             Line 93    lcd_data
   <= "00111100";
22 --      1-line mode, display on             Line 94    lcd_data
   <= "00110100";
23 --      1-line mode, display off            Line 95    lcd_data
   <= "00110000";
24 --      2-line mode, display off            Line 96    lcd_data
   <= "00111000";
25 --   Display ON/OFF
26 --      display on, cursor off, blink off   Line 104   lcd_data
   <= "00001100";
27 --      display on, cursor off, blink on    Line 105   lcd_data
   <= "00001101";
28 --      display on, cursor on, blink off    Line 106   lcd_data
   <= "00001110";
29 --      display on, cursor on, blink on     Line 107   lcd_data
   <= "00001111";
30 --      display off, cursor off, blink off  Line 108   lcd_data
   <= "00001000";
31 --      display off, cursor off, blink on   Line 109   lcd_data
   <= "00001001";
32 --      display off, cursor on, blink off   Line 110   lcd_data
   <= "00001010";
33 --      display off, cursor on, blink on    Line 111   lcd_data
   <= "00001011";
34 --   Entry Mode Set
35 --      increment mode, entire shift off    Line 127   lcd_data
   <= "00000110";
36 --      increment mode, entire shift on     Line 128   lcd_data
   <= "00000111";
37 --      decrement mode, entire shift off    Line 129   lcd_data
   <= "00000100";
38 --      decrement mode, entire shift on     Line 130   lcd_data
```

```
    <= "00000101";
39  --
40  -------------------------------------------------------------------
    -------------

41
42  LIBRARY ieee;
43  USE ieee.std_logic_1164.ALL;
44
45  ENTITY lcd IS
46    PORT(
47      clk        : IN    STD_LOGIC;  --system clock
48      reset_n    : IN    STD_LOGIC;  --active low reinitializes lcd
49      rw, rs, e, l_on  : OUT   STD_LOGIC;  --read/write,
    setup/data, and enable for lcd
50      lcd_data   : OUT   STD_LOGIC_VECTOR(7 DOWNTO 0); --data
    signals for lcd
51      line1_buffer : IN STD_LOGIC_VECTOR(127 downto 0); -- Data for
    the top line of the LCD
52      line2_buffer : IN STD_LOGIC_VECTOR(127 downto 0)); -- Data
    for the bottom line of the LCD
53  END lcd;
54
55  ARCHITECTURE controller OF lcd IS
56    TYPE CONTROL IS(power_up, initialize, RESETLINE, line1, line2,
    send);
57    SIGNAL    state      : CONTROL;
58    CONSTANT  freq       : INTEGER := 100; --system clock frequency
    in MHz
59    SIGNAL    ptr           : natural range 0 to 16 := 15; -- To
    keep track of what character we are up to
60    SIGNAL    line          : STD_LOGIC := '1';
61  BEGIN
62    PROCESS(clk)
63      VARIABLE clk_count : INTEGER := 0; --event counter for timing
64    BEGIN
65    IF(clk'EVENT and clk = '1') THEN
66      l_on<= '1';
67        CASE state IS
68
69          --wait 50 ms to ensure Vdd has risen and required LCD
    wait is met
70          WHEN power_up =>
71            IF(clk_count < (50000 * freq)) THEN    --wait 50 ms
72              clk_count := clk_count + 1;
73              state <= power_up;
74            ELSE                                   --power-up
    complete
75              clk_count := 0;
76              rs <= '0';
77              rw <= '0';
78              lcd_data <= "00110000";
79              state <= initialize;
80            END IF;
81
82          --cycle through initialization sequence
83          WHEN initialize =>
84            clk_count := clk_count + 1;
85            IF(clk_count < (10 * freq)) THEN       --function set
86              lcd_data <= "00110100";    --1-line mode, display on
```

```
87              e <= '1';
88              state <= initialize;
89            ELSIF(clk_count < (60 * freq)) THEN    --wait 50 us
90              lcd_data <= "00000000";
91              e <= '0';
92              state <= initialize;
93            ELSIF(clk_count < (70 * freq)) THEN    --display on/off
    control
94              lcd_data <= "00111100";      --display on, cursor
    off, blink off
95              e <= '1';
96              state <= initialize;
97            ELSIF(clk_count < (120 * freq)) THEN   --wait 50 us
98              lcd_data <= "00000000";
99              e <= '0';
100             state <= initialize;
101           ELSIF(clk_count < (130 * freq)) THEN   --display clear
102             lcd_data <= "00000001";
103             e <= '1';
104             state <= initialize;
105           ELSIF(clk_count < (2130 * freq)) THEN  --wait 2 ms
106             lcd_data <= "00000000";
107             e <= '0';
108             state <= initialize;
109           ELSIF(clk_count < (2140 * freq)) THEN  --entry mode set
110             lcd_data <= "00000110";      --increment mode, entire
    shift off
111             state <= initialize;
112           ELSIF(clk_count < (2200 * freq)) THEN  --wait 60 us
113             lcd_data <= "00000000";
114             e <= '0';
115             state <= initialize;
116           ELSE                                   --initialization
    complete
117             clk_count := 0;
118             state <= RESETLINE;
119           END IF;
120
121           WHEN resetline =>
122              ptr <= 16;
123              if line = '1' then
124                 lcd_data <= "10000000";
125                 rs <= '0';
126                 rw <= '0';
127                 clk_count := 0;
128                 state <= send;
129              else
130                lcd_data <= "11000000";
131                 rs <= '0';
132                 rw <= '0';
133                 clk_count := 0;
134                 state <= send;
135              end if;
136
137         WHEN line1 =>
138             line <= '1';
139             lcd_data <= line1_buffer(ptr*8 + 7 downto ptr*8);
140             rs <= '1';
141             rw <= '0';
```

```
142             clk_count := 0;
143             line <= '1';
144             state <= send;
145
146           WHEN line2 =>
147                 line <= '0';
148                 lcd_data <= line2_buffer(ptr*8 + 7 downto ptr*8);
149             rs <= '1';
150             rw <= '0';
151             clk_count := 0;
152             state <= send;
153
154         --send instruction to lcd
155         WHEN send =>
156               IF(clk_count < (50 * freq)) THEN  --do not exit for
    50us
157                   IF(clk_count < freq) THEN      --negative enable
158                     e <= '0';
159                   ELSIF(clk_count < (14 * freq)) THEN  --positive
    enable half-cycle
160                     e <= '1';
161                   ELSIF(clk_count < (27 * freq)) THEN  --negative
    enable half-cycle
162                     e <= '0';
163                   END IF;
164                   clk_count := clk_count + 1;
165                   state <= send;
166               ELSE
167                   clk_count := 0;
168                   if line = '1' then
169                        if ptr = 0 then
170                            line <= '0';
171                            state <= resetline;
172                        else
173                            ptr <= ptr - 1;
174                            state <= line1;
175                        end if;
176                   else
177                        if ptr = 0 then
178                            line <= '1';
179                            state <= resetline;
180                        else
181                            ptr <= ptr - 1;
182                            state <= line2;
183                        end if;
184                   end if;
185               END IF;
186
187       END CASE;
188
189       --reset
190       IF(reset_n = '0') THEN
191           state <= power_up;
192       END IF;
193
194     END IF;
195   END PROCESS;
196 END controller;
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;
4
5   entity main is
6       port(   clk, nrst, game_done, user_word_ready: in std_logic;
7               char: in std_logic_vector (7 downto 0);
8               get_word, game_over, start_game,ack_flag: out
    std_logic);
9   end main;
10
11  architecture arc of main is
12      type state_type is (w_start,s_ack, enter_code, w_user_word,
    s_start_game, w_game ,s_end);
13      signal st : state_type;
14      signal start: std_logic;
15      signal J_GO : std_logic;
16      signal K_GO : std_logic;
17  begin
18
19      -- States machine
20      process(clk, nrst) begin
21          if nrst = '0' then
22              st <= w_start;
23          elsif clk'event and clk = '1' then
24              case st is
25                  when w_start =>
26                      if (start = '1') then
27                          st <= s_ack;
28                      end if;
29                  when s_ack =>
30                      st <= enter_code;
31                  when enter_code =>
32                      st <= w_user_word;
33                  when w_user_word =>
34                      if (user_word_ready = '1') then
35                          st <= s_start_game;
36                      end if;
37                  when s_start_game =>
38                      st <= w_game;
39                  when w_game =>
40                      if game_done = '1' then
41                          st <= s_end;
42                      end if;
43                  when s_end =>
44                      st <= w_start;
45              end case;
46          end if;
47      end process;
48
49
50      -- Combinacional circuit to activate the start
51      -- signal when a #1 is received form keyboard
52      process(char)
53      begin
54          if st = w_start then
55              if char = x"31" then
56                  start <= '1';
57              else
```

```
58                  start <= '0';
59              end if;
60          end if;
61      end process;
62
63      -- JK flip-flop
64      process(clk,nrst,J_GO,K_GO)
65      begin
66          if nrst = '0' then
67              game_over <= '1';
68          elsif (clk'event and clk = '1') then
69              if J_GO = '1' then
70                  game_over <= '1';
71              elsif K_GO = '1' then
72                  game_over <= '0';
73              end if;
74          end if;
75      end process;
76
77  K_GO <= '1' when st = enter_code else '0';
78  J_GO <= '1' when st = s_end else '0';
79  get_word <= '1' when st = enter_code else '0';
80  ack_flag <= '1' when st = s_ack else '0';
81  start_game <= '1' when st = s_start_game else '0';
82
83  end arc;
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;
4
5   entity PS2_keyboard_interface is
6       port (
7           clk, nrst, scan_ready, ack_flag, new_code: in std_logic;
8           scan_code : in std_logic_vector(7 downto 0);
9           flag, in_read : out std_logic;
10          keycode : out std_logic_vector(7 downto 0)
11          );
12  end PS2_keyboard_interface;
13
14  architecture main of PS2_keyboard_interface is
15      type states is (w_newcode, w_scan_ready, load_reg, w_sc2, ack2
    , w_sc3, activate_flag);
16      signal st : states;
17      signal ld : std_logic;
18      signal set_flag : std_logic;
19      signal ascii : std_logic_vector(7 downto 0);
20  begin
21
22
23      -- 8-bit register
24      process(clk,ld,nrst)
25      begin
26          if nrst = '0' then
27              keycode <= x"00";
28          elsif (clk'event and clk = '1') then
29              if ld = '1' then
30                  keycode <= ascii;
31              end if;
32          end if;
33      end process;
34
35      -- JK flip-flop
36      process(clk, set_flag, ack_flag,nrst)
37      begin
38          if nrst = '0' then
39              flag <= '0';
40          elsif (clk'event and clk = '1') then
41              if set_flag = '1' then
42                  flag <= '1';
43              elsif ack_flag = '1' then
44                  flag <= '0';
45              end if;
46          end if;
47      end process;
48
49      -- State machine
50      process(clk,nrst)
51      begin
52          if(clk'event and clk = '1') then
53              if (nrst='0') then
54                  st <= w_newcode;
55              else
56                  case st is
57                      when w_newcode =>
58                          if (new_code='1') then
```

```
59                              st <= w_scan_ready;
60                          end if;
61                      when w_scan_ready =>
62                          if (scan_ready = '1') then
63                              st <= load_reg;
64                          end if;
65                      when load_reg =>
66                          st <= w_sc2;
67                      when w_sc2 =>
68                          if (scan_ready ='1') then
69                              st <= ack2;
70                          end if;
71                      when ack2 =>
72                          st <= w_sc3;
73                      when w_sc3 =>
74                          if scan_ready = '1' then
75                              st <= activate_flag;
76                          end if;
77                      when activate_flag =>
78                          st <= w_newcode;
79                  end case;
80              end if;
81          end if;
82      end process;
83
84  ld <= '1' when st = load_reg else '0';
85  in_read <= '1' when st = load_reg or st = ack2 or st =
    activate_flag else '0';
86  set_flag <= '1' when st = activate_flag else '0';
87
88  -- PS2 to ASCII system
89  process(scan_code)
90  begin
91      case scan_code is
92          WHEN x"29" => ascii <= x"20"; --space
93          WHEN x"66" => ascii <= x"08"; --backspace (BS control code)
94          WHEN x"0D" => ascii <= x"09"; --tab (HT control code)
95          WHEN x"5A" => ascii <= x"0D"; --enter (CR control code)
96          WHEN x"76" => ascii <= x"1B"; --escape (ESC control code)
97          WHEN x"1C" => ascii <= x"61"; --a
98          WHEN x"32" => ascii <= x"62"; --b
99          WHEN x"21" => ascii <= x"63"; --c
100         WHEN x"23" => ascii <= x"64"; --d
101         WHEN x"24" => ascii <= x"65"; --e
102         WHEN x"2B" => ascii <= x"66"; --f
103         WHEN x"34" => ascii <= x"67"; --g
104         WHEN x"33" => ascii <= x"68"; --h
105         WHEN x"43" => ascii <= x"69"; --i
106         WHEN x"3B" => ascii <= x"6A"; --j
107         WHEN x"42" => ascii <= x"6B"; --k
108         WHEN x"4B" => ascii <= x"6C"; --l
109         WHEN x"3A" => ascii <= x"6D"; --m
110         WHEN x"31" => ascii <= x"6E"; --n
111         WHEN x"44" => ascii <= x"6F"; --o
112         WHEN x"4D" => ascii <= x"70"; --p
113         WHEN x"15" => ascii <= x"71"; --q
114         WHEN x"2D" => ascii <= x"72"; --r
115         WHEN x"1B" => ascii <= x"73"; --s
116         WHEN x"2C" => ascii <= x"74"; --t
```

```
117           WHEN x"3C" => ascii <= x"75"; --u
118           WHEN x"2A" => ascii <= x"76"; --v
119           WHEN x"1D" => ascii <= x"77"; --w
120           WHEN x"22" => ascii <= x"78"; --x
121           WHEN x"35" => ascii <= x"79"; --y
122           WHEN x"1A" => ascii <= x"7A"; --z
123           WHEN x"45" => ascii <= x"30"; --0
124           WHEN x"16" => ascii <= x"31"; --1
125           WHEN x"1E" => ascii <= x"32"; --2
126           WHEN x"26" => ascii <= x"33"; --3
127           WHEN x"25" => ascii <= x"34"; --4
128           WHEN x"2E" => ascii <= x"35"; --5
129           WHEN x"36" => ascii <= x"36"; --6
130           WHEN x"3D" => ascii <= x"37"; --7
131           WHEN x"3E" => ascii <= x"38"; --8
132           WHEN x"46" => ascii <= x"39"; --9
133           WHEN x"52" => ascii <= x"27"; --'
134           WHEN x"41" => ascii <= x"2C"; --,
135           WHEN x"4E" => ascii <= x"2D"; ---
136           WHEN x"49" => ascii <= x"2E"; --.
137           WHEN x"4A" => ascii <= x"2F"; --/
138           WHEN x"4C" => ascii <= x"3B"; --;
139           WHEN x"55" => ascii <= x"3D"; --=
140           WHEN x"54" => ascii <= x"5B"; --[
141           WHEN x"5D" => ascii <= x"5C"; --\
142           WHEN OTHERS => NULL;
143       end case;
144   end process;
145
146   end;
```

```
1   LIBRARY IEEE;
2   USE IEEE.STD_LOGIC_1164.ALL;
3   USE IEEE.STD_LOGIC_ARITH.ALL;
4   USE IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6   ENTITY PS2_keyboard IS
7       PORT(   keyboard_clk, keyboard_data, clock_25Mhz ,
8               reset, in_read : IN STD_LOGIC;
9               scan_code : OUT STD_LOGIC_VECTOR( 7 DOWNTO 0 );
10              scan_ready : OUT STD_LOGIC);
11  END PS2_keyboard;
12
13  ARCHITECTURE a OF PS2_keyboard IS
14      SIGNAL INCNT                    : STD_LOGIC_VECTOR( 3 DOWNTO 0
     );
15      SIGNAL SHIFTIN                  : STD_LOGIC_VECTOR( 8 DOWNTO 0
     );
16      SIGNAL READ_CHAR                : STD_LOGIC;
17      SIGNAL INFLAG, ready_set        : STD_LOGIC;
18      SIGNAL keyboard_clk_filtered    : STD_LOGIC;
19      SIGNAL filter                   : STD_LOGIC_VECTOR( 7 DOWNTO 0
     );
20
21
22  BEGIN
23      PROCESS ( in_read, ready_set )
24      BEGIN
25          IF in_read = '1' THEN
26              scan_ready <= '0';
27          ELSIF ready_set'EVENT AND ready_set = '1' THEN
28              scan_ready <= '1';
29          END IF;
30      END PROCESS;
31          --This process filters the raw clock signal coming from the
32          -- keyboard using a shift register and two AND gates
33
34  Clock_filter:
35      PROCESS
36          BEGIN
37          WAIT UNTIL clock_25Mhz'EVENT AND clock_25Mhz = '1';
38          filter ( 6 DOWNTO 0 ) <= filter( 7 DOWNTO 1 ) ;
39          filter( 7 ) <= keyboard_clk;
40          IF filter = "11111111" THEN
41              keyboard_clk_filtered <= '1';
42          ELSIF filter = "00000000" THEN
43              keyboard_clk_filtered <= '0';
44          END IF;
45      END PROCESS Clock_filter;
46          --This process reads in serial scan code data coming from
    the keyboard
47
48      PROCESS
49      BEGIN
50          WAIT UNTIL (keyboard_clk_filtered'EVENT AND
    KEYBOARD_CLK_filtered = '1');
51          IF reset = '0' THEN
52              INCNT <= "0000";
53              READ_CHAR <= '0';
54          ELSE
```

```
55              IF KEYBOARD_DATA = '0' AND READ_CHAR = '0' THEN
56                  READ_CHAR <= '1';
57                  ready_set <= '0';
58              ELSE
59                      -- Shift in next 8 data bits to assemble a
    scan code
60                  IF READ_CHAR = '1' THEN
61                      IF INCNT < "1001" THEN
62                          INCNT <= INCNT + 1 ;
63                          SHIFTIN( 7 DOWNTO 0 ) <= SHIFTIN( 8 DOWNTO
     1 );
64                          SHIFTIN( 8 ) <= KEYBOARD_DATA;
65                          ready_set <= '0';
66                                  -- End of scan code character, so
    set flags and exit loop
67                      ELSE
68                          scan_code <= SHIFTIN( 7 DOWNTO 0 );
69                          READ_CHAR <='0';
70                          ready_set <= '1';
71                          INCNT <= "0000";
72                      END IF;
73                  END IF;
74              END IF;
75          END IF;
76      END PROCESS;
77  END a;
78
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;
4
5   entity register_bank_guessing_char is
6       port(   clk50, nrst,char_flag, new_char : in std_logic;
7               char : in std_logic_vector (7 downto 0);
8               out_char : out std_logic_vector (7 downto 0);
9               key_read, bank_ready : out std_logic);
10  end register_bank_guessing_char;
11
12  architecture arc of register_bank_guessing_char is
13      type state_type is (s_wait, w_charflag, loadreg, s_end);
14      signal st : state_type;
15      signal buff_char : std_logic_vector (7 downto 0);
16      signal J_BR : std_logic;
17      signal K_BR : std_logic;
18      signal ld   : std_logic;
19  begin
20
21      -- States machine
22      process(clk50, nrst) begin
23          if nrst = '0' then
24              st <= s_wait;
25          elsif clk50'event and clk50 = '1' then
26              case st is
27                  when s_wait =>
28                      if (new_char = '1') then
29                          st <= w_charflag;
30                      end if;
31                  when w_charflag =>
32                      if (char_flag = '1') then
33                          if char /= x"0D" then
34                              st <= loadreg;
35                          else
36                              st <= s_end;
37                          end if;
38                      end if;
39                  when loadreg =>
40                      st <= w_charflag;
41                  when s_end =>
42                      st <= s_wait;
43              end case;
44          end if;
45      end process;
46
47
48      -- JK flip-flop
49      process(clk50,nrst,new_char,J_BR)
50      begin
51          if nrst = '0' then
52              bank_ready <= '1';
53          elsif (clk50'event and clk50 = '1') then
54              if J_BR = '1' then
55                  bank_ready <= '1';
56              elsif new_char = '1' then
57                  bank_ready <= '0';
58              end if;
59          end if;
```

```
60        end process;
61
62        -- Load input char to register
63        process(clk50,ld,nrst)
64        begin
65            if nrst = '0' then
66                buff_char <= x"00";
67            elsif (clk50'event and clk50 = '1') then
68                if ld = '1' then
69                    buff_char <= char;
70                end if;
71            end if;
72        end process;
73
74    key_read <= '1' when st = w_charflag else '0';
75    J_BR <= '1' when st = s_end else '0';
76    ld <= '1' when st = loadreg else '0';
77    out_char <= buff_char;
78
79    end arc;
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;
4
5   entity register_bank_opp_inc_word is
6       port(   clk50, nrst, f_new : in std_logic;
7               new_word: in std_logic_vector (127 downto 0);
8               word: out std_logic_vector (127 downto 0);
9               bank_ready : out std_logic);
10  end register_bank_opp_inc_word;
11
12  architecture arc of register_bank_opp_inc_word is
13      type state_type is (s0, s1, s2, s3);
14      signal st :  state_type;
15      signal buff_word : std_logic_vector (127 downto 0);
16  begin
17      process(clk50, nrst) begin
18          if nrst = '0' then
19              bank_ready <= '1';
20              buff_word <= (others => '0');
21              st <= s0;
22          elsif clk50'event and clk50 = '1' then
23              case st is
24                  when s0 =>
25                      if f_new = '1' then
26                          st <= s1;
27                      end if;
28                  when s1 =>
29                      st <= s2;
30                      bank_ready <= '0';
31                  when s2 =>
32                      st <= s3;
33                      buff_word <= new_word;
34                  when s3 =>
35                      st <= s0;
36                      bank_ready <= '1';
37              end case;
38          end if;
39      end process;
40  word <= buff_word;
41  end arc;
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;
4
5   entity register_bank_opponent_cv is
6       port(   clk50, nrst, load_cv : in std_logic;
7               op_cv : in std_logic_vector (15 downto 0);
8               cv_out : out std_logic_vector (15 downto 0);
9               key_read, bank_ready : out std_logic);
10  end register_bank_opponent_cv;
11
12  architecture arc of register_bank_opponent_cv is
13      type state_type is (s_wait, loadreg, s_end);
14      signal st : state_type;
15      signal J_BR : std_logic;
16      signal K_BR : std_logic;
17      signal ld   : std_logic;
18  begin
19
20      -- States machine
21      process(clk50, nrst) begin
22          if nrst = '0' then
23              st <= s_wait;
24          elsif clk50'event and clk50 = '1' then
25              case st is
26                  when s_wait =>
27                      if (load_cv = '1') then
28                          st <= loadreg;
29                      end if;
30                  when loadreg =>
31                      st <= s_end;
32                  when s_end =>
33                      st <= s_wait;
34              end case;
35          end if;
36      end process;
37
38
39      -- JK flip-flop
40      process(clk50,nrst,load_cv,J_BR)
41      begin
42          if nrst = '0' then
43              bank_ready <= '1';
44          elsif (clk50'event and clk50 = '1') then
45              if J_BR = '1' then
46                  bank_ready <= '1';
47              elsif load_cv = '1' then
48                  bank_ready <= '0';
49              end if;
50          end if;
51      end process;
52
53      -- Load input char to register
54      process(clk50,ld,nrst)
55      begin
56          if nrst = '0' then
57              cv_out <= x"0000";
58          elsif (clk50'event and clk50 = '1') then
59              if ld = '1' then
```

```
60                      cv_out <= op_cv;
61                  end if;
62              end if;
63          end process;
64
65      J_BR <= '1' when st = s_end else '0';
66      ld <= '1' when st = loadreg else '0';
67
68      end arc;
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;
4
5   entity register_bank_user_word is
6       port(   clk50, nrst,char_flag, new_word: in std_logic;
7               char: in std_logic_vector (7 downto 0);
8               user_word: out std_logic_vector (127 downto 0);
9               key_read, bank_ready : out std_logic);
10  end register_bank_user_word;
11
12  architecture arc of register_bank_user_word is
13      type state_type is (s_wait,s_marranada, s_new_word, w_charflag
    , loadreg, s_end);
14      signal st : state_type;
15      signal buff : std_logic_vector (127 downto 0);
16      signal J_BR : std_logic;
17      signal K_BR : std_logic;
18      signal ld   : std_logic;
19      signal clr  : std_logic;
20  begin
21
22      -- States machine
23      process(clk50, nrst) begin
24          if nrst = '0' then
25              st <= s_wait;
26          elsif clk50'event and clk50 = '1' then
27              case st is
28                  when s_wait =>
29                      if (new_word = '1') then
30                          st <= s_marranada;
31                      end if;
32                  when s_marranada =>
33                      st <= s_new_word;
34                  when s_new_word =>
35                      st <= w_charflag;
36                  when w_charflag =>
37                      if (char_flag = '1') then
38                          if char /= x"0D" then
39                              st <= loadreg;
40                          else
41                              st <= s_end;
42                          end if;
43                      end if;
44                  when loadreg =>
45                      st <= w_charflag;
46                  when s_end =>
47                      st <= s_wait;
48              end case;
49          end if;
50      end process;
51
52      -- Slicing 128-bit register
53      process(clk50,ld,nrst,clr)
54      begin
55          if nrst = '0' or clr = '1' then
56              buff <= x"00000000000000000000000000000000";
57          elsif (clk50'event and clk50 = '1') then
58              if clr = '1' then
```

```
59                  buff <= x"00000000000000000000000000000000";
60              elsif ld = '1' then
61                  if char /= x"31" then
62                      buff(127 downto 120) <= buff(119 downto 112);
63                      buff(119 downto 112) <= buff(111 downto 104);
64                      buff(111 downto 104) <= buff(103 downto 96);
65                      buff(103 downto 96) <= buff(95 downto 88);
66                      buff(95 downto 88) <= buff(87 downto 80);
67                      buff(87 downto 80) <= buff(79 downto 72);
68                      buff(79 downto 72) <= buff(71 downto 64);
69                      buff(71 downto 64) <= buff(63 downto 56);
70                      buff(63 downto 56) <= buff(55 downto 48);
71                      buff(55 downto 48) <= buff(47 downto 40);
72                      buff(47 downto 40) <= buff(39 downto 32);
73                      buff(39 downto 32) <= buff(31 downto 24);
74                      buff(31 downto 24) <= buff(23 downto 16);
75                      buff(23 downto 16) <= buff(15 downto 8);
76                      buff(15 downto 8) <= buff(7 downto 0);
77                      buff(7 downto 0) <= char;
78                  end if;
79              end if;
80          end if;
81      end process;
82
83
84      -- JK flip-flop
85      process(clk50,nrst,new_word,J_BR)
86      begin
87          if nrst = '0' then
88              bank_ready <= '0';
89          elsif (clk50'event and clk50 = '1') then
90              if J_BR = '1' then
91                  bank_ready <= '1';
92              elsif K_BR = '1' then
93                  bank_ready <= '0';
94              end if;
95          end if;
96      end process;
97
98  key_read <= '1' when st = w_charflag or st = s_marranada else '0';
99  J_BR <= '1' when st = s_end else '0';
100 K_BR <= '1' when st = s_new_word else '0';
101 ld <= '1' when st = loadreg else '0';
102 clr <= '1' when st = s_new_word else '0';
103 user_word <= buff;
104
105 end arc;
```

```
1   --Serial receiver.
2   --Default baud rate is 56000 bps from a 50MHz clock.
3   --Version 1.2 october 2016.
4
5   library ieee;
6   use ieee.std_logic_1164.all;
7
8   entity rx_module is
9   port( clk, nrst, rx, data_read : in std_logic;
10        rx_data : out std_logic_vector(7 downto 0);
11        rx_new, rx_data_lost  : out std_logic );
12  end rx_module;
13
14  architecture main of rx_module is
15
16  type state_type is (rx_idle, rx_start, rx_shift, rx_stop);
17  signal state : state_type;
18
19  signal bit_count : integer range 0 to 7;   -- bit count within
    the received data word
20  --constant max : integer range 0 to 7 := 4;     -- set to 4 only
    for simulation purposes
21  constant max : integer range 0 to 8191 := 5208;  --
    50M/5208~9600, /1302~38400, /892~56000, etc.
22  signal baud : integer range 0 to max; -- rx counter at the baud
    rate
23
24  -- Added from the old version
25  signal set_rx_new: std_logic; -- to set rx_new to one.
26  signal rx_new_internal: std_logic;  -- replica of rx_new to be
    read.
27
28  begin
29
30  --------- RECEIVER_FSM ---------
31  RX_FSM: process(clk, nrst) begin
32
33  if nrst = '0' then
34      state <= rx_idle;
35      baud <= 0;
36      --rx_new <= '0';
37      rx_data_lost <= '0';
38
39  elsif clk'event and clk = '1' then
40      case state is
41      when rx_idle => bit_count <= 0;
42                      --rx_new <= '0';
43                      if rx = '0' then
44                          baud <= 0;
45                          state <= rx_start;
46                      end if;
47      when rx_start => if (baud = (max-1)/2) then
48                          baud <= 0;
49                          if rx = '0' then
50                              state <= rx_shift;
51                              bit_count <= 0;
52                          else
53                              state <= rx_idle;
54                          end if;
```

```
55                      else
56                          baud <= baud + 1;
57                      end if;
58      when rx_shift => if (baud = max) then
59                          baud <= 0;
60                          bit_count <= bit_count+1;
61                          rx_data(bit_count) <= rx;

62                          if (bit_count = 7) then
63                              bit_count <= 0;
64                              state <= rx_stop;
65                              --rx_new <='1';
66                              -- Added from the old version:
67                              if (rx_new_internal = '1') then
68                                  rx_data_lost <= '1';
69                              end if;
70                          end if;
71                       else
72                          baud <= baud + 1;
73                       end if;
74      when rx_stop => --rx_new <= '0';
75                      if (baud = max) then
76                          baud <= 0;
77                          bit_count <= 0;
78                          state <= rx_idle;
79                      else
80                          baud <= baud + 1;
81                      end if;
82      when others =>
83      end case;
84  end if;
85  end process;
86
87  -- Added from the old version:
88
89  -- Signal set to activate rx_new.
90
91  --set_rx_new <= '1' when state = rx_shift and bit_count = 7 and
    baud = max else '0';
92  set_rx_new <= '1' when state = rx_stop and baud = max else '0';
93
94  -- Process to hand shake with the user.
95  -- J K flip flop to set and reset rx_new.
96
97  process(clk, nrst) begin
98
99      if (nrst = '0') then
100         rx_new_internal <= '0';
101     elsif clk'event and clk = '1' then
102         if (set_rx_new = '1') then
103             rx_new_internal <= '1';
104         elsif (data_read = '1') then
105             rx_new_internal <= '0';
106         end if;
107     end if;
108 end process;
109
110 rx_new <= rx_new_internal;
111
```

```
112    end;
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3
4   entity rx_protocol is
5       port (
6           clk50, nrst, frame_read, rx_new : in std_logic;
7           rx_data : in std_logic_vector(7 downto 0);
8           data_read, new_frame : out std_logic;
9           loser, winner, new_cv, new_op_char, new_op_cv : out
    std_logic;
10          op_char : out std_logic_vector (7 downto 0);
11          op_cv, cv_ack : out std_logic_vector (15 downto 0));
12  end rx_protocol;
13
14  architecture main of rx_protocol is
15      type state_type is (pr_ini, pr_w1, pr_r1, pr_w2, pr_r2,
    pr_f_type, pr_s_loser, pr_s_winner, pr_w_ready1, pr_r_ready1,
16                          pr_w_ready2, pr_r_ready2, pr_s_ready,
    pr_w_cvack1, pr_r_cvack1, pr_w_cvack2, pr_r_cvack2,
17                          pr_s_cvack, pr_w_gc, pr_r_gc, pr_s_gc);
18      signal state : state_type;
19       -- Definition of the diferent FRAME TYPE
20      constant FT_READY : std_logic_vector(7 downto 0) := x"A1";
21      constant FT_CV_ACK : std_logic_vector(7 downto 0) := x"B1";
22      constant FT_GC : std_logic_vector(7 downto 0) := x"C1";
23      constant FT_LOSER : std_logic_vector(7 downto 0) := x"D1";
24      constant FT_WINNER : std_logic_vector(7 downto 0) := x"D2";
25      constant FRAME_ID : std_logic_vector(7 downto 0) := x"AA";
26      -- Definition of the registers in the process unit
27      signal frame_type: std_logic_vector(7 downto 0);
28      -- Signals to activate the output flags
29      signal set_LOSER, set_WINNER, set_NEW_CV, set_NEW_OP_CHAR,
    set_NEW_OP_CV : std_logic;
30      signal set_new_frame : std_logic;
31
32  begin
33
34  PROTOCOL_FSM : process (clk50, nrst) begin
35      if nrst = '0' then
36          state <= pr_ini;
37
38      elsif clk50'EVENT and clk50 = '1' then
39          case state is
40              when pr_ini =>
41                  state <= pr_w1;
42
43              -- Waiting for frame_ID to arrive
44              when pr_w1 =>
45                  if rx_new = '1' then state <= pr_r1;
46                  end if;
47
48              -- Frame ID arrived. We do not check it..
49              -- Data_read should be activated within a
    combinational system
50              when pr_r1 =>
51                  state <= pr_w2;
52
53              -- Waiting for FRAME TYPE to arive
54              when pr_w2 =>
```

```
55                  if rx_new = '1' then state <= pr_r2;
56                  end if;
57              -- FRAME TYPE ARRIVED.
58              -- Data Read should be activated within a
    combinational system
59              when pr_r2 =>
60                  frame_type <= rx_data;
61                  state <= pr_f_type;
62
63              -- Checking frame type. If type contains data the
    system read the extra bytes.
64              when pr_f_type =>
65                  if frame_type = FT_READY then state <= pr_w_ready1;
66                  elsif frame_type = FT_CV_ACK then state <=
    pr_w_cvack1;
67                  elsif frame_type = FT_GC then state <= pr_w_gc;
68                  elsif frame_type = FT_LOSER then state <=
    pr_s_loser;
69                  elsif frame_type = FT_WINNER then state <=
    pr_s_winner;
70                  end if;
71
72              when pr_s_loser =>
73                  state <= pr_ini;
74
75              when pr_s_winner =>
76                  state <= pr_ini;
77
78              when pr_w_ready1 =>
79                  if rx_new = '1' then state <= pr_r_ready1;
80                  end if;
81
82              when pr_r_ready1 =>
83                  op_cv(15 downto 8) <= rx_data;
84                  state <= pr_w_ready2;
85
86              when pr_w_ready2 =>
87                  if rx_new = '1' then state <= pr_r_ready2;
88                  end if;
89
90              when pr_r_ready2 =>
91                  op_cv(7 downto 0) <= rx_data;
92                  state <= pr_s_ready;
93
94              when pr_s_ready =>
95                  state <= pr_w1;
96
97              when pr_w_cvack1 =>
98                  if rx_new = '1' then state <= pr_r_cvack1;
99                  end if;
100
101             when pr_r_cvack1 =>
102                 cv_ack(15 downto 8) <= rx_data;
103                 state <= pr_w_cvack2;
104
105             when pr_w_cvack2 =>
106                 if rx_new = '1' then state <= pr_r_cvack2;
107                 end if;
108
```

```
109                 when pr_r_cvack2 =>
110                     cv_ack(7 downto 0) <= rx_data;
111                     state <= pr_s_cvack;
112
113                 when pr_s_cvack =>
114                     state <= pr_w1;
115
116                 when pr_w_gc =>
117                     if rx_new = '1' then state <= pr_r_gc;
118                     end if;
119
120                 when pr_r_gc =>
121                     op_char <= rx_data;
122                     state <= pr_s_gc;
123
124                 when pr_s_gc =>
125                     state <= pr_w1;
126
127                 when others =>
128                     end case;
129         end if;
130     end process;
131
132     -- Combinational system for the signals that activate
133     set_LOSER <= '1' when state = pr_s_loser else '0';
134     set_WINNER <= '1' when state = pr_s_winner else '0';
135     set_NEW_CV <= '1' when state = pr_s_ready else '0';
136     set_NEW_OP_CHAR <= '1' when state = pr_s_gc else '0';
137     set_NEW_OP_CV <= '1' when state = pr_s_cvack else '0';
138     data_read <= '1' when state = pr_r1 or state = pr_r2 or state =
        pr_r_ready1 or
139         state = pr_r_ready2 or state = pr_r_cvack1 or state =
        pr_r_cvack2 or state = pr_r_gc else '0';
140
141     JK: process(clk50, nrst) begin
142             if nrst = '0' then
143                 loser <= '0';
144                 winner <= '0';
145                 new_cv <= '0';
146                 new_op_char <= '0';
147                 new_op_cv <= '0';
148                 new_frame <= '0';
149             elsif clk50'EVENT and clk50 = '1' then
150                 if set_LOSER = '1' then loser <= '1'; new_frame <= '1'
    ; end if;
151                 if set_WINNER = '1' then winner <= '1'; new_frame <= '
    1'; end if;
152                 if set_NEW_CV = '1' then new_cv <= '1'; new_frame <= '
    1'; end if;
153                 if set_NEW_OP_CHAR = '1' then new_op_char <= '1';
    new_frame <= '1'; end if;
154                 if set_NEW_OP_CV = '1' then new_op_cv <= '1';
    new_frame <= '1'; end if;
155                 if frame_read = '1' then
156                     loser <= '0';
157                     winner <= '0';
158                     new_cv <= '0';
159                     new_op_char <= '0';
160                     new_op_cv <= '0';
```

```
161                     new_frame <= '0';
162                 end if;
163             end if;
164         end process;
165
166     end;
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3
4   entity tx_module is
5       port (
6           clk, nrst, send : in std_logic;
7           tx_data : in std_logic_vector(7 downto 0);
8           tx, tx_empty : out std_logic
9           );
10  end tx_module;
11
12
13  architecture main of tx_module is
14      type state_type is (tx_idle, tx_start, tx_shift, tx_stop);
15      signal state : state_type;
16      signal bit_count : integer range 0 to 7; -- bit count within
    the received data word
17      --constant max : integer range 0 to 7 := 4; -- set to 4 only
    for simulation purposes
18      constant max : integer range 0 to 8191 := 5208; --
    50M/5208~9600, /1302~38400, /892~56000, etc.
19      signal baud : integer range 0 to max; -- rx counter at the
    baud rate
20  begin
21
22  TX_FSM : process (clk, nrst) begin
23      if nrst = '0' then
24          state <= tx_idle;
25          baud <= 0;
26          --- Added from original version:
27          tx <= '1';
28          tx_empty <= '1';
29      elsif clk'EVENT and clk = '1' then
30          case state is
31              when tx_idle => bit_count <= 0;
32                  tx <= '1';
33                  baud <= 0;
34                  tx_empty <= '1';
35                  if send = '1' then
36                          tx_empty <= '0';
37                          state <= tx_start;
38                  end if;
39              when tx_start => tx <= '0';
40                  if (baud = max) then
41                      baud <= 0;
42                      state <= tx_shift;
43                  else
44                      baud <= baud + 1;
45                  end if;
46              when tx_shift => tx <= tx_data(bit_count);
47                      if (baud = max) then
48                          baud <= 0;
49                          bit_count <= bit_count + 1;
50                          if (bit_count = 7) then
51                              bit_count <= 0;
52                              state <= tx_stop;
53                          end if;
54                      else
55                          baud <= baud + 1;
```

```
56                          end if;
57                  when tx_stop => tx <= '1';
58                          if (baud = max) then
59                              baud <= 0;
60                              state <= tx_idle;
61                          else
62                              baud <= baud + 1;
63                          end if;
64                  when others =>
65                      end case;
66          end if;
67      end process;
68      end;
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;
4
5   entity tx_protocol is
6       port(   clk, nrst, send_ready, send_op_cv_ack: in std_logic;
7               cv, op_cv_ack : in std_logic_vector (15 downto 0);
8               guessing_char : in std_logic_vector (7 downto 0);
9               send_gc, send_loser, send_winner, tx_empty: in
    std_logic;
10              tx_data : out std_logic_vector (7 downto 0);
11              send, ready_to_tx: out std_logic);
12  end tx_protocol;
13
14  architecture arc of tx_protocol is
15      type state_type is (w_frame_code, s_rdy, w_tx_empty1,
    s_frame_id, w_tx_empty2,
16                          s_frame_type, w_tx_empty3, s_decision,
    s_cv1, w_tx_empty4,
17                          s_cv2, s_cvack1, w_tx_empty5, s_cvack2,
    s_gc, s_end);
18      signal st : state_type;
19      signal J_RDY, J_FCR : std_logic;
20      signal K_RDY, k_FCR : std_logic;
21      signal frame_code : std_logic_vector(7 downto 0);
22      signal frame_code_received : std_logic;
23  begin
24
25      -- States machine
26      process(clk, nrst) begin
27          if nrst = '0' then
28              st <= w_frame_code;
29              tx_data <= x"00";
30          elsif clk'event and clk = '1' then
31              case st is
32                  when w_frame_code =>
33                      if (frame_code_received = '1') then
34                          st <= s_rdy;
35                      end if;
36                  when s_rdy =>
37                      st <= w_tx_empty1;
38                  when w_tx_empty1 =>
39                      if (tx_empty = '1') then
40                          st <= s_frame_id;
41                      end if;
42                  when s_frame_id =>
43                      tx_data <= x"AA";
44                      st <= w_tx_empty2;
45                  when w_tx_empty2 =>
46                      if (tx_empty = '1') then
47                          st <= s_frame_type;
48                      end if;
49                  when s_frame_type =>
50                      tx_data <= frame_code;
51                      st <= w_tx_empty3;
52                  when w_tx_empty3 =>
53                      if (tx_empty = '1') then
54                          st <= s_decision;
55                      end if;
```

```
56                  when s_decision =>
57                      if (frame_code = x"A1") then
58                          st <= s_cv1;
59                      elsif (frame_code = x"B1") then
60                          st <= s_cvack1;
61                      elsif (frame_code = x"C1") then
62                          st <= s_gc;
63                      else
64                          st <= s_end;
65                      end if;
66                  when s_cv1 =>
67                      tx_data <= cv(15 downto 8);
68                      st <= w_tx_empty4;
69                  when w_tx_empty4 =>
70                      if (tx_empty = '1') then
71                          st <= s_cv2;
72                      end if;
73                  when s_cv2 =>
74                      tx_data <= cv(7 downto 0);
75                      st <= s_end;
76                  when s_cvack1 =>
77                      tx_data <= op_cv_ack(15 downto 8);
78                      st <= w_tx_empty5;
79                  when w_tx_empty5 =>
80                      if (tx_empty = '1') then
81                          st <= s_cvack2;
82                      end if;
83                  when s_cvack2 =>
84                      tx_data <= op_cv_ack(7 downto 0);
85                      st <= s_end;
86                  when s_gc =>
87                      tx_data <= guessing_char;
88                      st <= s_end;
89                  when s_end =>
90                      st <= w_frame_code;
91              end case;
92          end if;
93      end process;
94
95
96      -- JK flip-flop
97      process(clk,nrst,J_RDY,K_RDY)
98      begin
99          if nrst = '0' then
100             ready_to_tx <= '1';
101         elsif (clk'event and clk = '1') then
102             if J_RDY = '1' then
103                 ready_to_tx <= '1';
104             elsif K_RDY = '1' then
105                 ready_to_tx <= '0';
106             end if;
107         end if;
108     end process;
109
110     -- Combinational that converts the input into the
    corresponding frame code.
111     process(clk,nrst, frame_code, frame_code_received)
112     begin
113         if nrst = '0' then
```

```
114                 frame_code_received <= '0';
115             elsif (clk'event and clk = '1') then
116                 if (send_ready = '1') then
117                     frame_code <= x"A1";
118                     frame_code_received <= '1';
119                 elsif (send_op_cv_ack = '1') then
120                     frame_code <= x"B1";
121                     frame_code_received <= '1';
122                 elsif (send_gc = '1') then
123                     frame_code <= x"C1";
124                     frame_code_received <= '1';
125                 elsif (send_loser = '1') then
126                     frame_code <= x"D1";
127                     frame_code_received <= '1';
128                 elsif (send_winner = '1') then
129                     frame_code <= x"D2";
130                     frame_code_received <= '1';
131                 else
132                     frame_code_received <= '0';
133                 end if;
134             end if;
135         end process;
136
137
138     K_RDY <= '1' when st = s_rdy else '0';
139     J_RDY <= '1' when st = s_end or st = w_frame_code else '0';
140     send <= '1' when st = s_frame_id or st = s_frame_type or st =
        s_cv1 or st = s_cv2 or st = s_cvack1 or st = s_cvack2 or st = s_gc
         else '0';
141
142     end arc;
```

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;
4   use ieee.numeric_std.all;
5
6   entity user_interface is
7       port(
8           clk, nrst, activate, game_over : in std_logic;
9           game_status : in std_logic_vector(1 downto 0);
10          lifes : in std_logic_vector(3 downto 0);
11          LR: out std_logic_vector(16 downto 0);
12          LG: out std_logic_vector(7 downto 0);
13          word_ready_out, game_status_out : out std_logic;
14          sev_seg : out std_logic_vector(29 downto 0);
15          lost, winner : in std_logic
16      );
17  end user_interface;
18
19  architecture main of user_interface is
20      type state_type is (s0_wait, s_wait_game, s_wait_status, s_lr,
     s_lg, s_game);
21      signal state : state_type;
22      signal lifesU, lifesT : std_logic_vector(4 downto 0);
23      --signal leds : std_logic_vector(16 downto 0);
24  begin
25
26      process(clk)
27      begin
28
29          if (nrst = '0') then
30              LR <= "00000000000000000";
31              LG <= "00000000";
32              sev_seg <= (others => '0');
33              word_ready_out <= '0';
34              game_status_out <= '0';
35              state <= s0_wait;
36          elsif(clk'event and clk = '1') then
37              case state is
38                  when s0_wait =>
39                      --if(activate = '1') then
40                          state <= s_wait_game;
41                      --end if;
42                      --lifesU <= (others => '0');
43                      --lifesT<= (others => '0');
44                      --word_ready_out <= '1';
45                  when s_wait_game =>
46                      if (game_over = '0') then
47                          state <= s_wait_status;
48                      else
49                          LR <= "00000000000000000";
50                          LG <= "00000000";
51                          state <= s0_wait;
52                      end if;
53                  when s_wait_status =>
54                      if (game_status = "00" or winner ='1') then
55                          state <= s_lg;
56                      elsif (game_status = "01" or lost = '1') then
57                          state <= s_lr;
58                      elsif (game_status ="10") then
```

```
59                      state <= s_game;
60                  end if;
61
62                  if (lifes = "00000" ) then --0
63                      lifesU <= "00000";
64                      lifesT <= "00000";
65                      --leds <= "0000000000000000";
66                  elsif (lifes = "00001") then --1
67                      lifesU <= "00001";
68                      lifesT <= "00000";
69                      --leds <= "0000000000000001";
70                  elsif (lifes = "00010") then --2
71                      lifesU <= "00010";
72                      lifesT <= "00000";
73                      --leds <= "0000000000000011";
74                  elsif (lifes = "00011") then --3
75                      lifesU <= "00011";
76                      lifesT <= "00000";
77                      --leds <= "0000000000000111";
78                  elsif (lifes = "00100") then --4
79                      lifesU <= "00100";
80                      lifesT <= "00000";
81                      --leds <= "0000000000001111";
82                  elsif (lifes = "00101") then --5
83                      lifesU <= "00101";
84                      lifesT <= "00000";
85                      --leds <= "0000000000011111";
86                  elsif (lifes = "00110") then --6
87                      lifesU <= "00110";
88                      lifesT <= "00000";
89                      --leds <= "0000000000111111";
90                  elsif (lifes = "00111") then --7
91                      lifesU <= "00111";
92                      lifesT <= "00000";
93                      --leds <= "0000000001111111";
94                  elsif (lifes = "01000") then --8
95                      lifesU <= "01000";
96                      lifesT <= "00000";
97                      --leds <= "0000000011111111";
98                  elsif (lifes = "01001") then --9
99                      lifesU <= "01001";
100                     lifesT <= "00000";
101                     --leds <= "0000000111111111";
102                 elsif (lifes = "01010") then --10
103                     lifesU <= "00000";
104                     lifesT <= "00001";
105                     --leds <= "0000001111111111";
106                 elsif (lifes = "01011") then --11
107                     lifesU <= "00001";
108                     lifesT <= "00001";
109                     --leds <= "0000011111111111";
110                 elsif (lifes = "01100") then --12
111                     lifesU <= "00010";
112                     lifesT <= "00001";
113                     --leds <= "0000111111111111";
114                 elsif (lifes = "01101") then --13
115                     lifesU <= "00011";
116                     lifesT <= "00001";
117                     --leds <= "0001111111111111";
```

```
118                     elsif (lifes = "01110") then --14
119                         lifesU <= "00100";
120                         lifesT <= "00001";
121                         --leds <= "0001111111111111";
122                     elsif (lifes = "01111") then --15
123                         lifesU <= "00101";
124                         lifesT <= "00001";
125                         --leds <= "0011111111111111";
126                     elsif (lifes = "10000") then --16
127                         lifesU <= "00110";
128                         lifesT <= "00001";
129                         --leds <= "0111111111111111";
130                     else
131                         lifesU <= "11111";
132                         lifesT <= "11111";
133                     end if;
134                 when s_lr =>
135                     --game_status_out <= '0';
136                     LR <= "1111111111111111";
137                     sev_seg(19 downto 0) <= "10100101011011010111";
138                     state <= s0_wait;
139                     sev_seg(29 downto 25) <= lifesT;
140                     sev_seg(24 downto 20) <= lifesU;
141                 when s_lg =>
142                     --game_status_out <= '1';
143                     LG <= "11111111";
144                     LR <= "00000000000000000";
145                     sev_seg(19 downto 0) <= "10000100011001010011";
146                     state <= s0_wait;
147                     sev_seg(29 downto 25) <= lifesT;
148                     sev_seg(24 downto 20) <= lifesU;
149                 when s_game =>
150                     sev_seg(29 downto 25) <= lifesT;
151                     sev_seg(24 downto 20) <= lifesU;
152                     sev_seg(19 downto 0) <= (others => '1');
153                     word_ready_out <= '1';
154                     --LR <= leds;
155                     state <= s0_wait;
156             end case;
157         end if;
158 end process;
159 end;
```

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_arith.all;
4
5  entity hex_disps is
6    port ( char0, char1, char2, char3, char4, char5 : in std_logic_vector (4 downto 0);
7           HEX0, HEX1, HEX2, HEX3, HEX4, HEX5 : out std_logic_vector (6 downto 0));
8  end hex_disps ;
9
10 architecture behav_display of hex_disps is
11
12   component BCD7seg
13     port ( num  : in std_logic_vector (4 downto 0);
14            HEX  : out std_logic_vector (6 downto 0));
15   end component;
16
17 begin
18
19  display0: BCD7seg
20    port map(char0,HEX0);
21  display1: BCD7seg
22    port map(char1,HEX1);
23  display2: BCD7seg
24    port map(char2,HEX2);
25  display3: BCD7seg
26    port map(char3,HEX3);
27  display4: BCD7seg
28    port map(char4,HEX4);
29  display5: BCD7seg
30    port map(char5,HEX5);
31
32 end behav_display;
33
```

```
1  library work;
2  library ieee;
3  use ieee.std_logic_1164.all;
4
5  entity BCD7seg is
6     port( num : in std_logic_vector (4 downto 0);
7           HEX  : out std_logic_vector (6 downto 0)
8            );
9  end BCD7seg;
10
11 architecture behaviour of BCD7seg is
12
13 begin
14
15         HEX(6 downto 0) <=   "1000000" when num(4 downto 0) = "00000" else  --0
16                              "1111001" when num(4 downto 0) = "00001" else --1
17                              "0100100" when num(4 downto 0) = "00010" else --2
18                              "0110000" when num(4 downto 0) = "00011" else --3
19                              "0011001" when num(4 downto 0) = "00100" else --4
20                              "0010010" when num(4 downto 0) = "00101" else --5
21                              "0000010" when num(4 downto 0) = "00110" else --6
22                              "1111000" when num(4 downto 0) = "00111" else --7
23                              "0000000" when num(4 downto 0) = "01000" else --8
24                              "0010000" when num(4 downto 0) = "01001" else --9
25                              "1000011" when num(4 downto 0) = "10000" else --W
26                              "1100001" when num(4 downto 0) = "10001" else --W
27                              "1111001" when num(4 downto 0) = "10010" else --I
28                              "1001000" when num(4 downto 0) = "10011" else --N
29                              "1000111" when num(4 downto 0) = "10100" else --L
30                              "1000000" when num(4 downto 0) = "10101" else --O
31                              "0010010" when num(4 downto 0) = "10110" else --S
32                              "0000110" when num(4 downto 0) = "10111" else --E
33                              "1111111";
34
35
36 end behaviour;
37
```